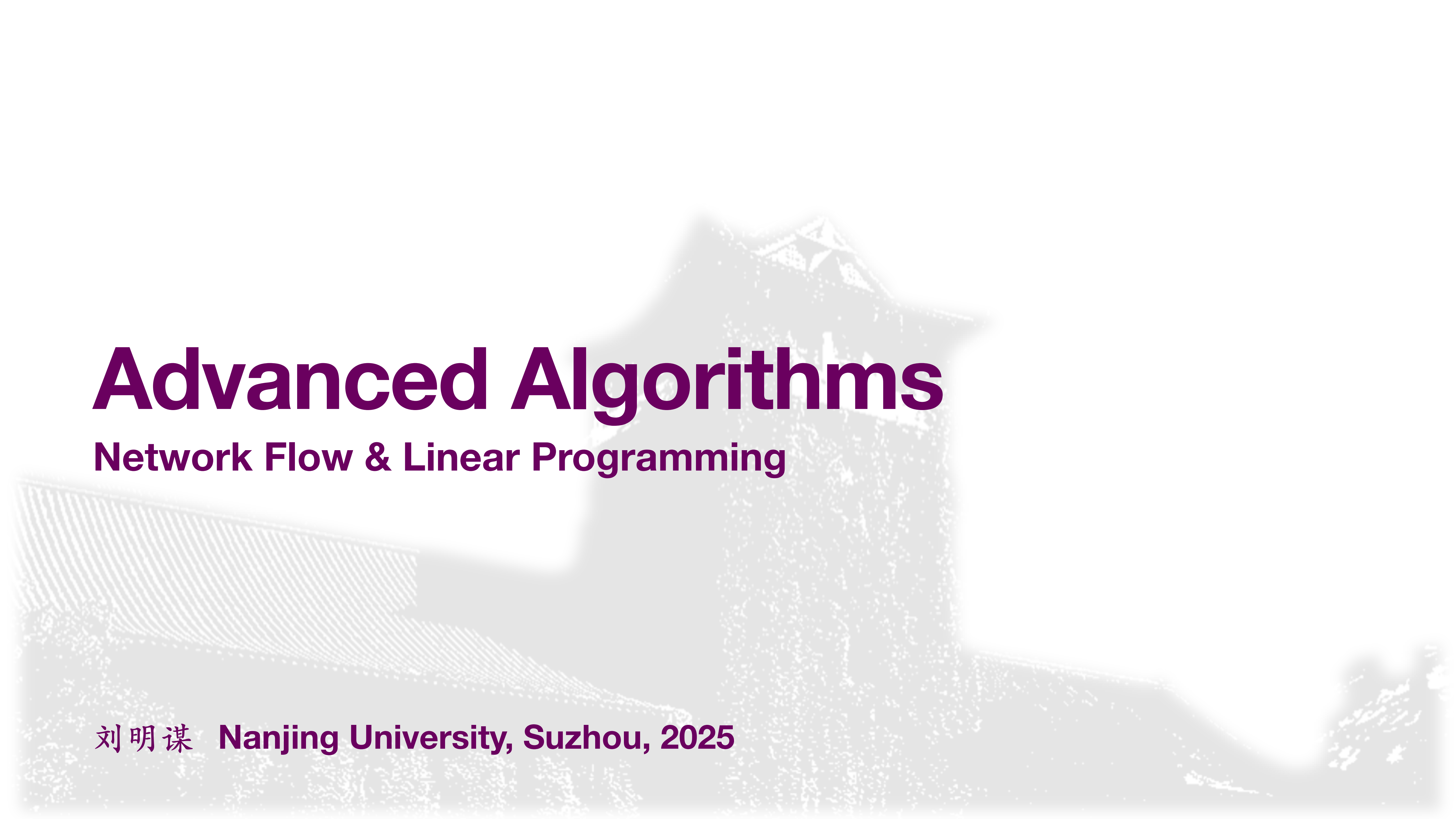


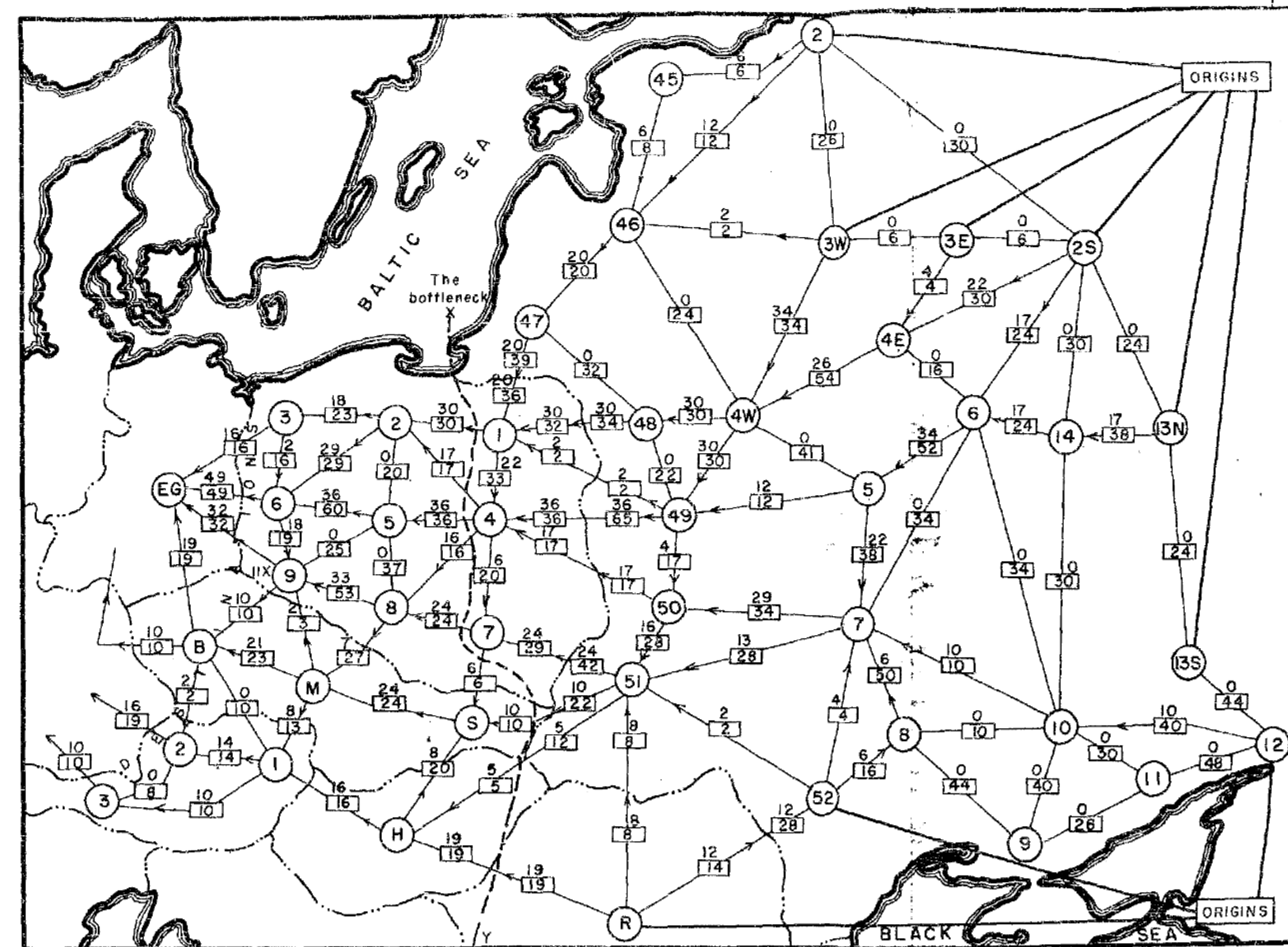


Advanced Algorithms

Network Flow & Linear Programming

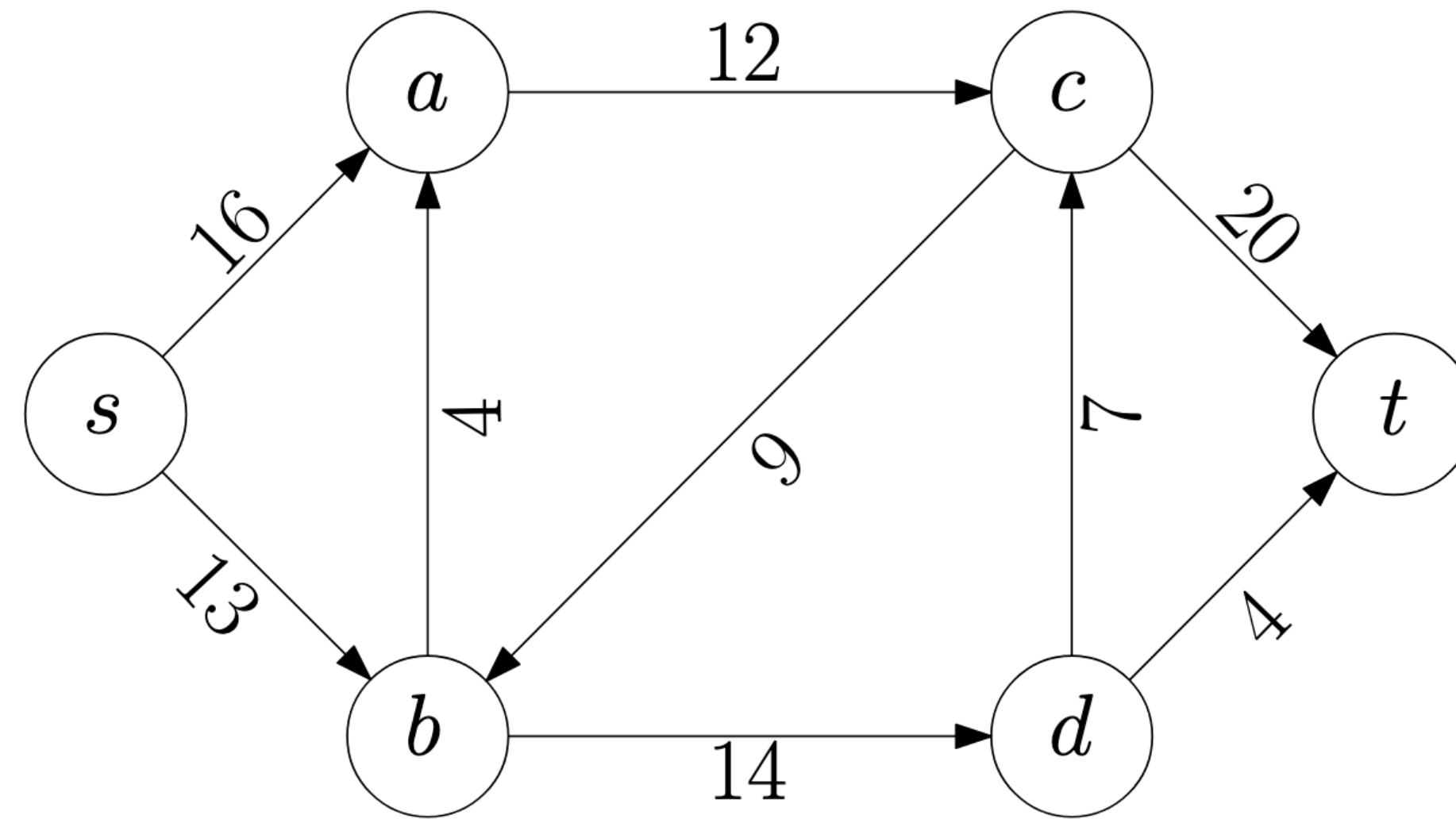
刘明谋 Nanjing University, Suzhou, 2025

Network Flow



Graph with Capacity

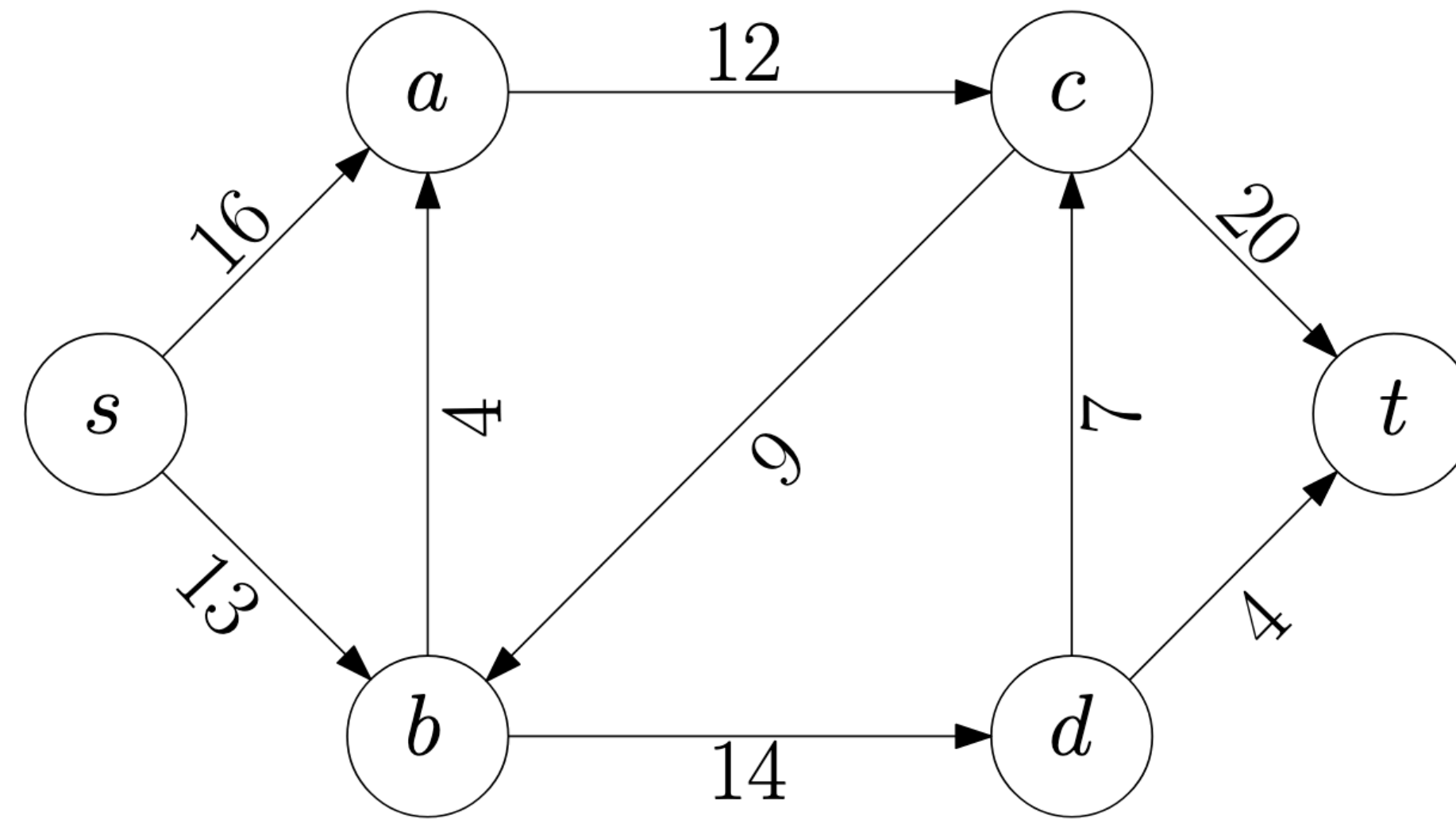
Flow



- Digraph $G = (V, E)$: a set of vertices V and a set of directed edges E
- Edge capacity $c_e \in \mathbb{R}^+$ for edge $e \in E$
- Flow function $f: E \rightarrow \mathbb{R}^+$
 - Valid flow: $\forall e \in E, 0 \leq f(e) \leq c_e$ & $\forall v \in V, \sum_{e \in \delta_{in}(v)} f(e) = \sum_{e \in \delta_{out}(v)} f(e)$

Flow network

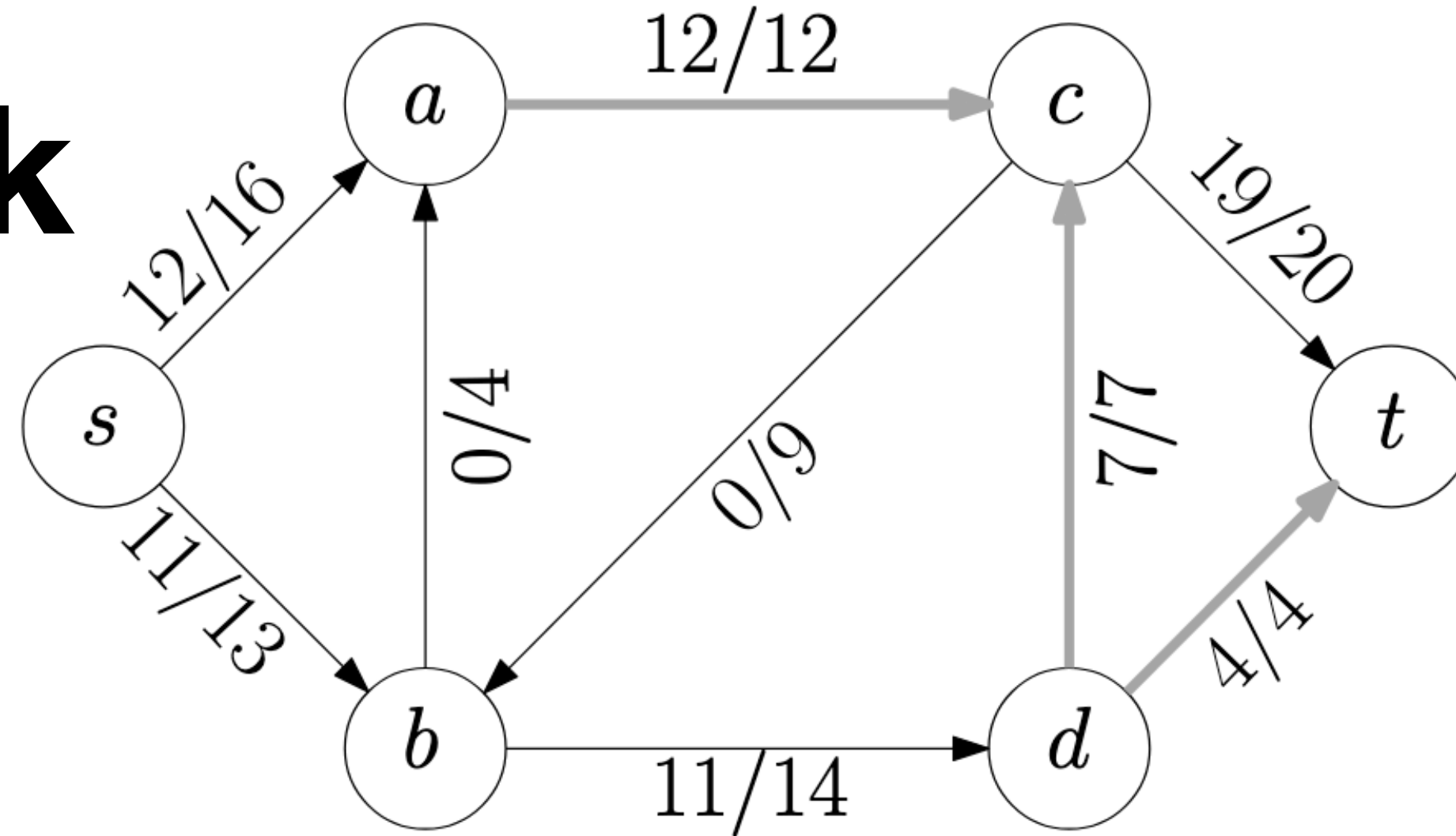
s-t flow



- Digraph $G = (V, E)$: a set of vertices V and a set of directed edges E
- Edge capacity $c_e \in \mathbb{R}^+$ for edge $e \in E$
- Flow function $f: E \rightarrow \mathbb{R}^+$, source $s \in V$, sink $t \in V$
 - Valid flow: $\forall e \in E, 0 \leq f(e) \leq c_e$ & $\forall v \in V \setminus \{s, t\}, \sum_{e \in \delta_{in}(v)} f(e) = \sum_{e \in \delta_{out}(v)} f(e)$

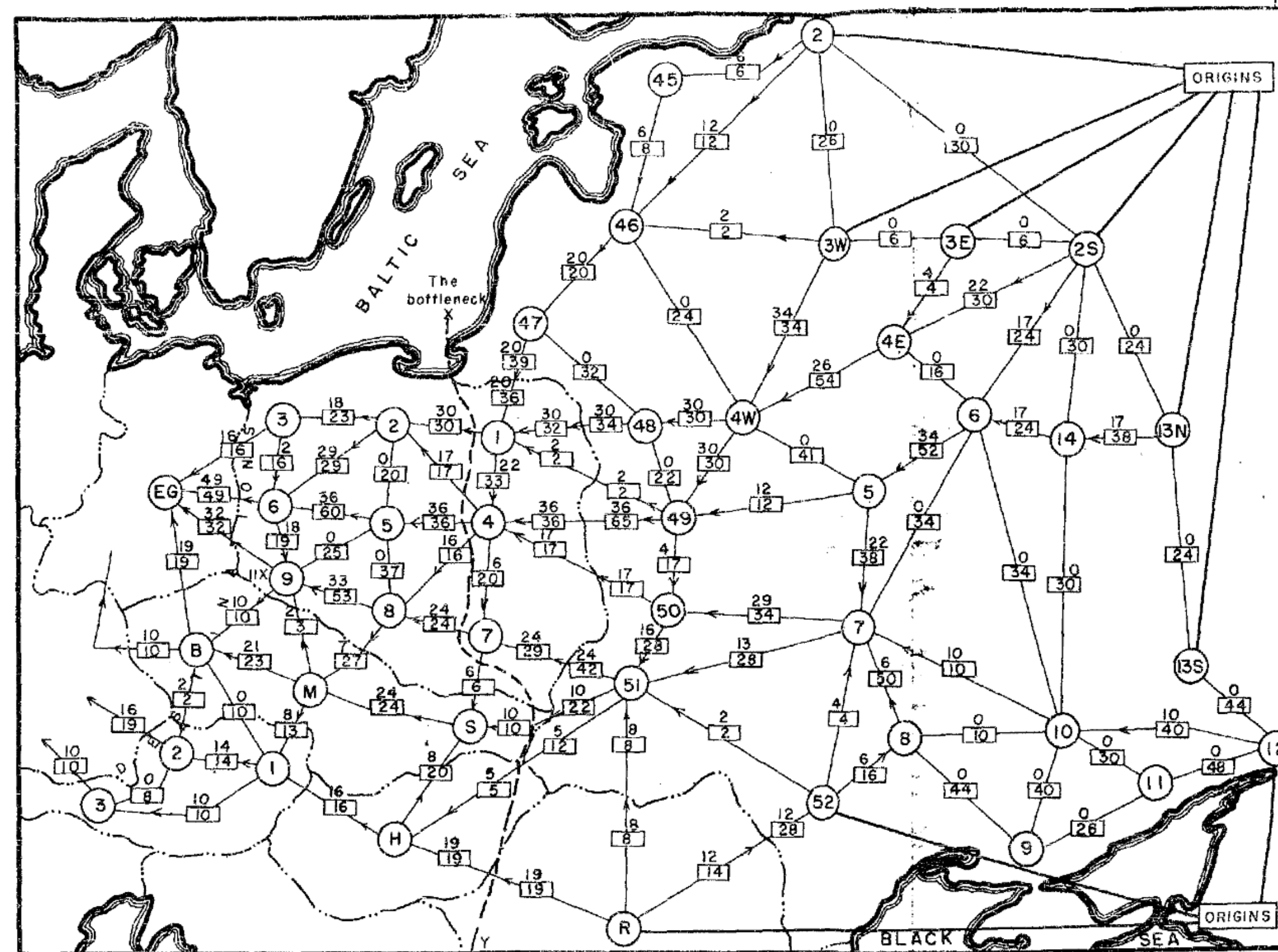
Flow network

Maximum flow



- Digraph $G = (V, E)$: a set of vertices V and a set of directed edges E
- Edge capacity $c_e \in \mathbb{R}^+$ for edge $e \in E$
- Flow function $f: E \rightarrow \mathbb{R}^+$, source $s \in V$, sink $t \in V$
 - Valid flow: $\forall e \in E, 0 \leq f(e) \leq c_e$ & $\forall v \in V \setminus \{s, t\}, \sum_{e \in \delta_{in}(v)} f(e) = \sum_{e \in \delta_{out}(v)} f(e)$
- Maximum flow problem: find a flow f maximizes $\sum_{e \in \delta_{out}(s)} f(e)$

Army transportation



Assumption:

Entire network available for east-west traffic (no allowance for civilian or economic traffic)

Results:

- 163,000 tons per day can be delivered from points of origin to destinations.
- 147,000 tons per day can be delivered without using Austrian lines.
- 152,000 tons per day can be delivered into Germany by all lines.
- 126,000 tons per day can be delivered into East Germany without using Austrian lines.

SECRET RM-1573
10-24-55
-33-

Fig. 7 — Traffic pattern: entire network available

Legend:

— International boundary

(8) Railway operating division

← 12 → Capacity: 12 each way per day. Required flow of 9 per day toward destinations (in direction of arrow) with equivalent number of returning trains in opposite direction

All capacities in trains
√1000's of tons each way per day

Origins: Divisions 2, 3W, 3E, 2S, 13N, 13S, 12, 52 (USSR), and Roumania

Destinations: Divisions 3, 6, 9 (Poland); B (Czechoslovakia); and 2, 3 (Austria)

Alternative destinations: Germany or East Germany

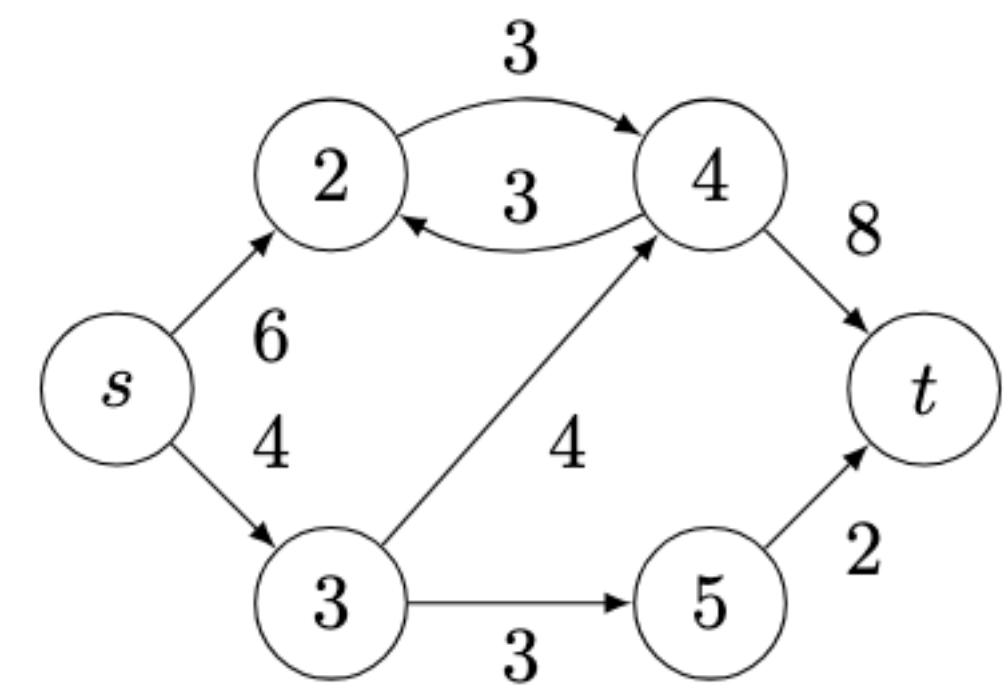
Note IIX at Division 9, Poland

SECRET

RM 1573-7

Flow decomposition

Maximum flow consists of paths



- Theorem: Any s - t flow can be decomposed into cycles and s - t paths.
- Constructive proof:

– While s has out flow: exists valid s - t path since $\sum_{e \in \delta_{out}(s)} f(e) = \sum_{e \in \delta_{in}(t)} f(e)$

‣ find s - t path $P \subseteq E$; let $\delta(P) \triangleq \min_{e \in P} f_e$; update $\forall e \in P, f_e \leftarrow f_e - \delta(P)$

– While exists flow: exists flow cycles since $\sum_{e \in \delta_{in}(v)} f(e) = \sum_{e \in \delta_{out}(v)} f(e), \forall v$

‣ find flow cycle $P \subseteq E$, remove it as well

Augmenting path

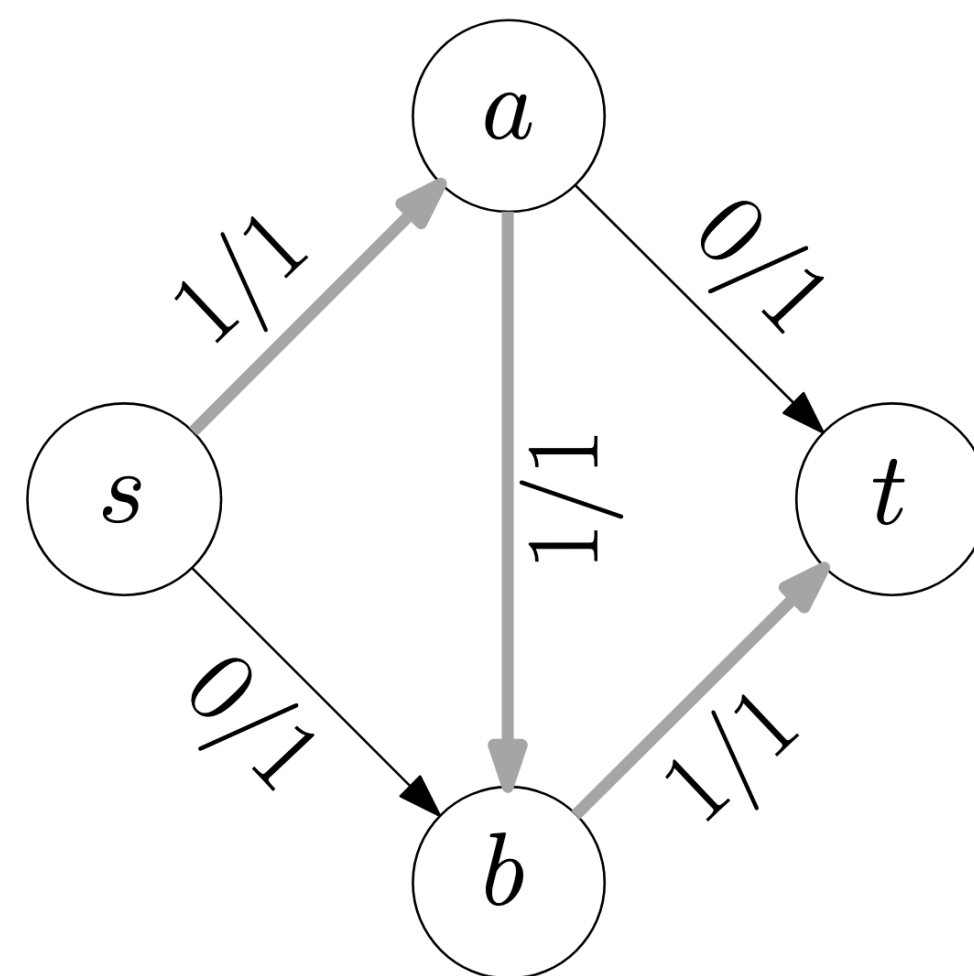
Maximum flow consists of paths

- Theorem: Any s - t flow can be decomposed into cycles and s - t paths.

Greedy: let $\delta(P) \triangleq \min_{e \in P} c_e - f_e$ be residual capacity

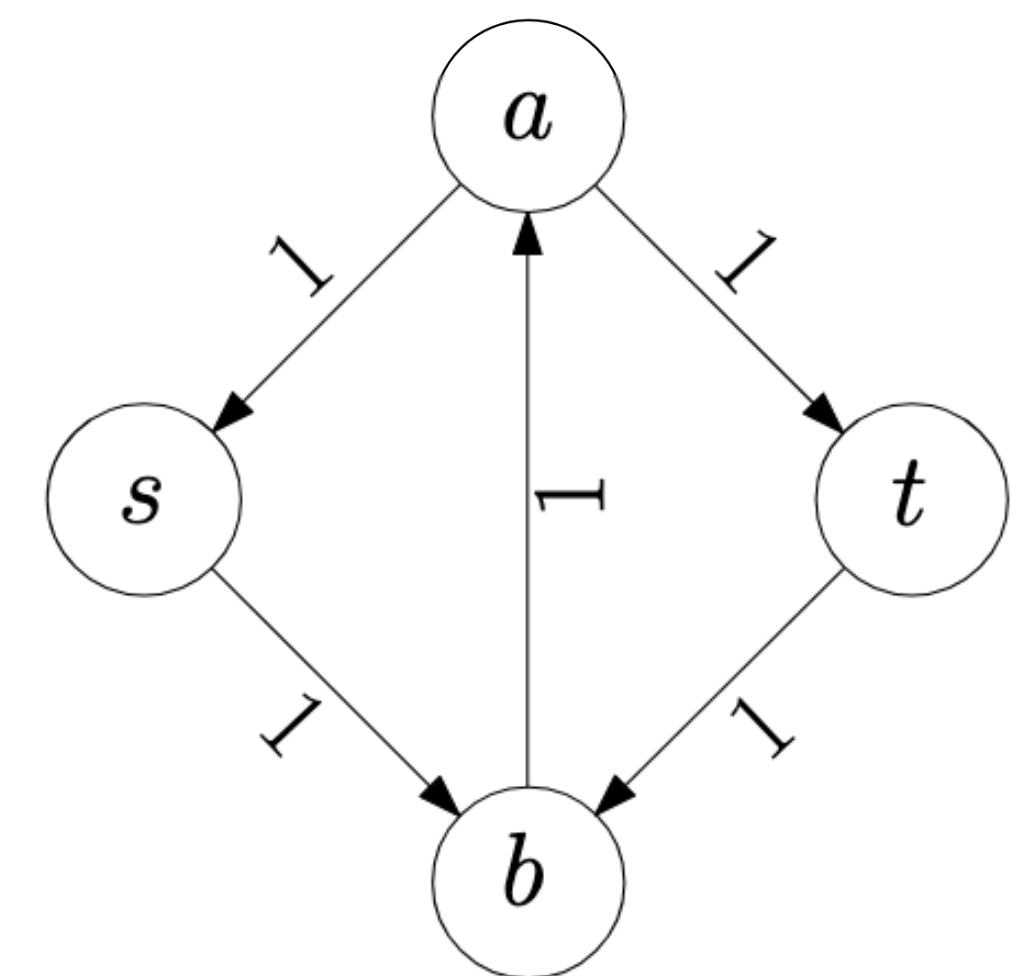
While exists s - t path P with $\delta(P) > 0$:
increase the flow in P by $\delta(P)$

- Any issue?



Observation:
implicit augmenting path

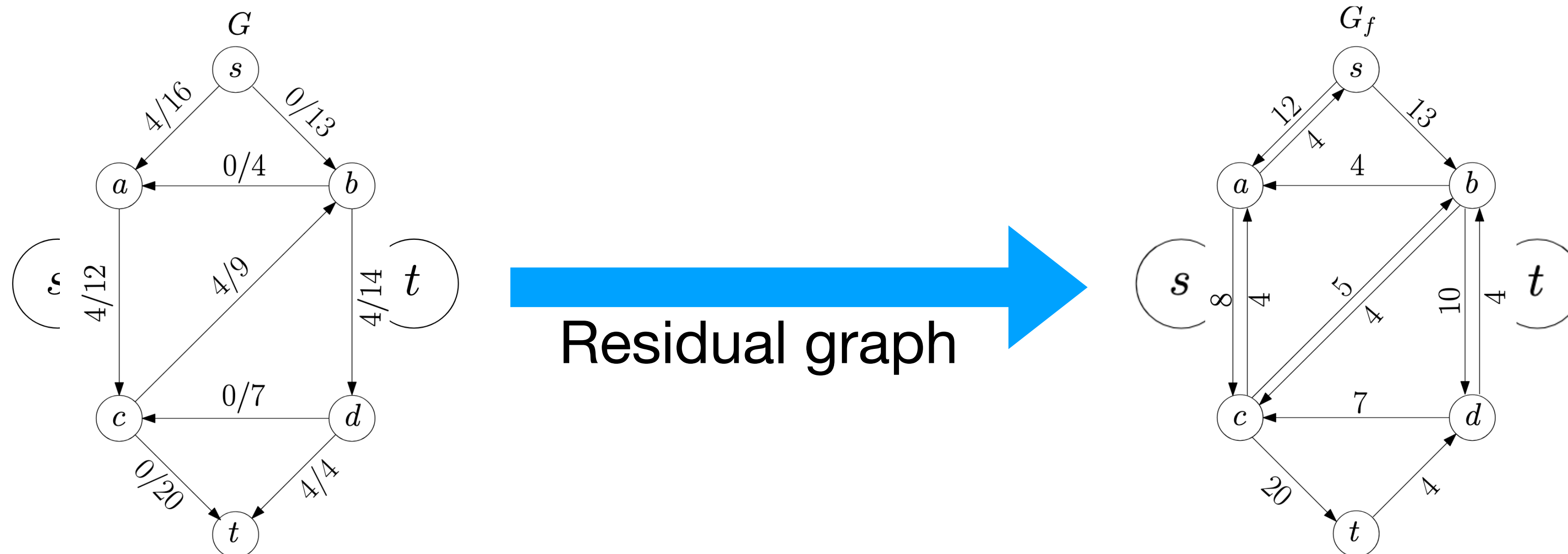
Residual graph



Augmenting path

Fixing with residual graph

- Theorem: Any s - t flow can be decomposed into cycles and s - t paths.
- **Residual graph** $G_f = (V, E')$ of $G = (V, E)$ with capacity c and flow f :
 - for each $e = (u, v) \in E$ with $c_e > f_e$, add e to E' with $c'_e = c_e - f_e$
 - for each $e = (u, v) \in E$ with $f_e > 0$, add (v, u) to E' with $c'_{e'} = f_e$

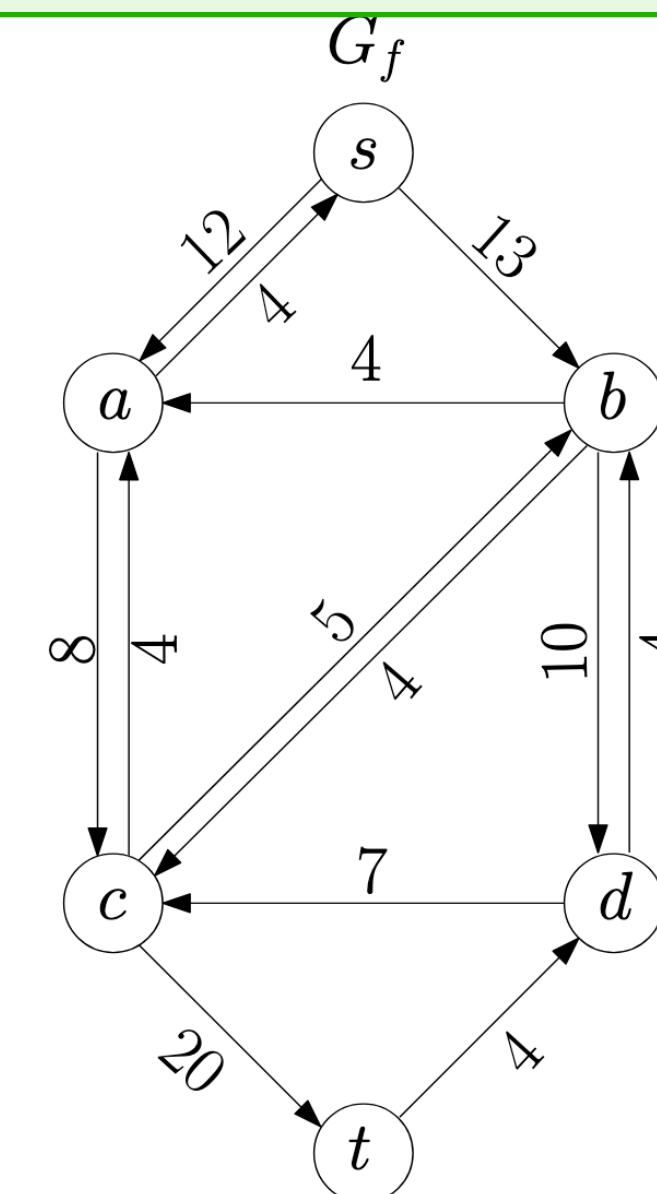
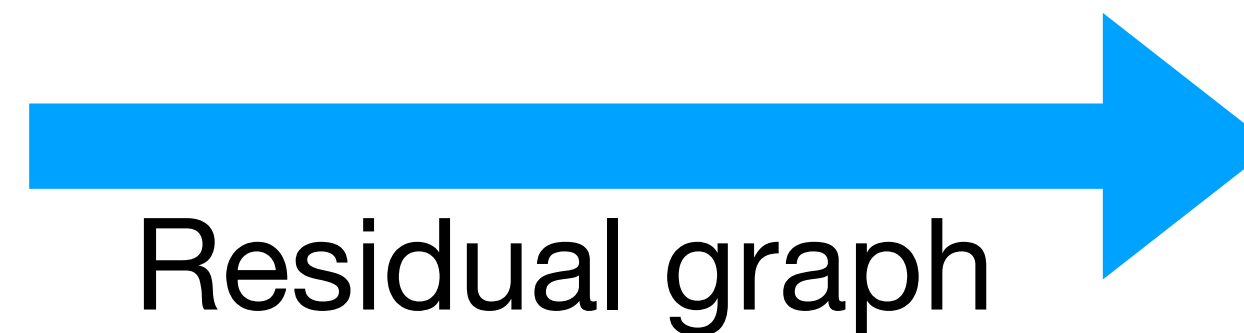
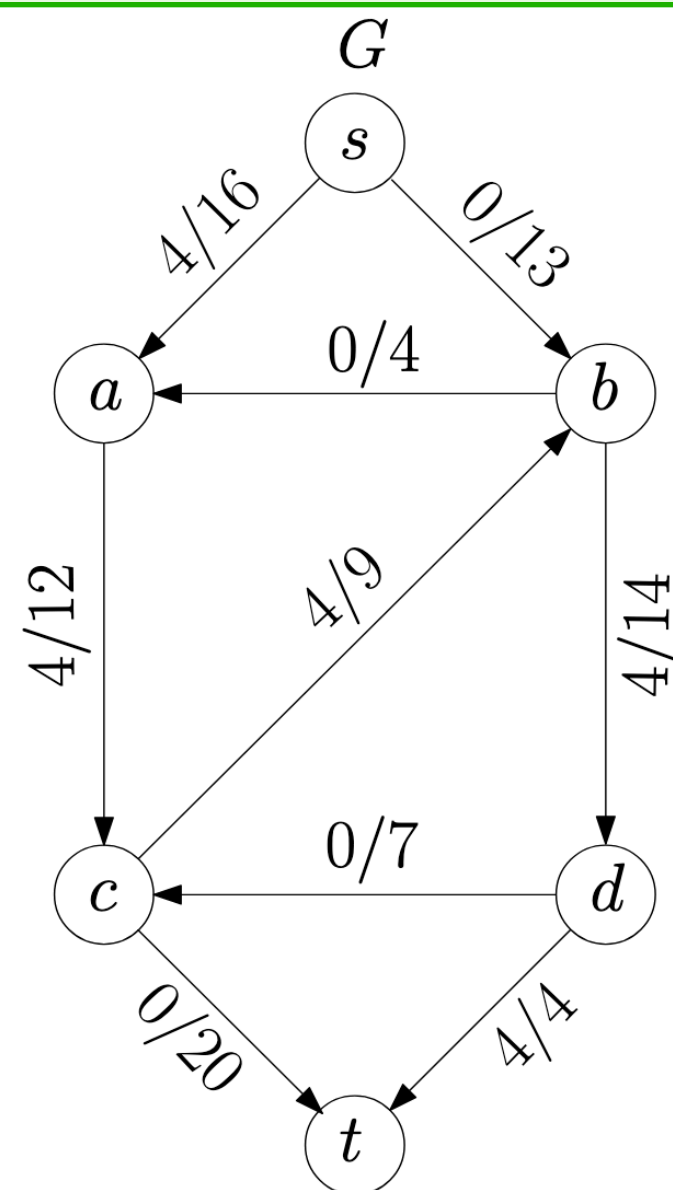


Augmenting path

Fixing with residual graph

- Theorem: Any s - t flow can be decomposed into cycles and s - t paths.

Ford-Fulkerson: let $\delta(P)$ be residual capacity for path P
While exists s - t path $P \in G^f$ with $\delta(P) > 0$:
increase the flow f in P by $\delta(P)$



Ford-Fulkerson: let $\delta(P)$ be residual capacity for path P
While exists s - t path $P \in G^f$ with $\delta(P) > 0$:
increase the flow f in P by $\delta(P)$

The following statements are equivalent:

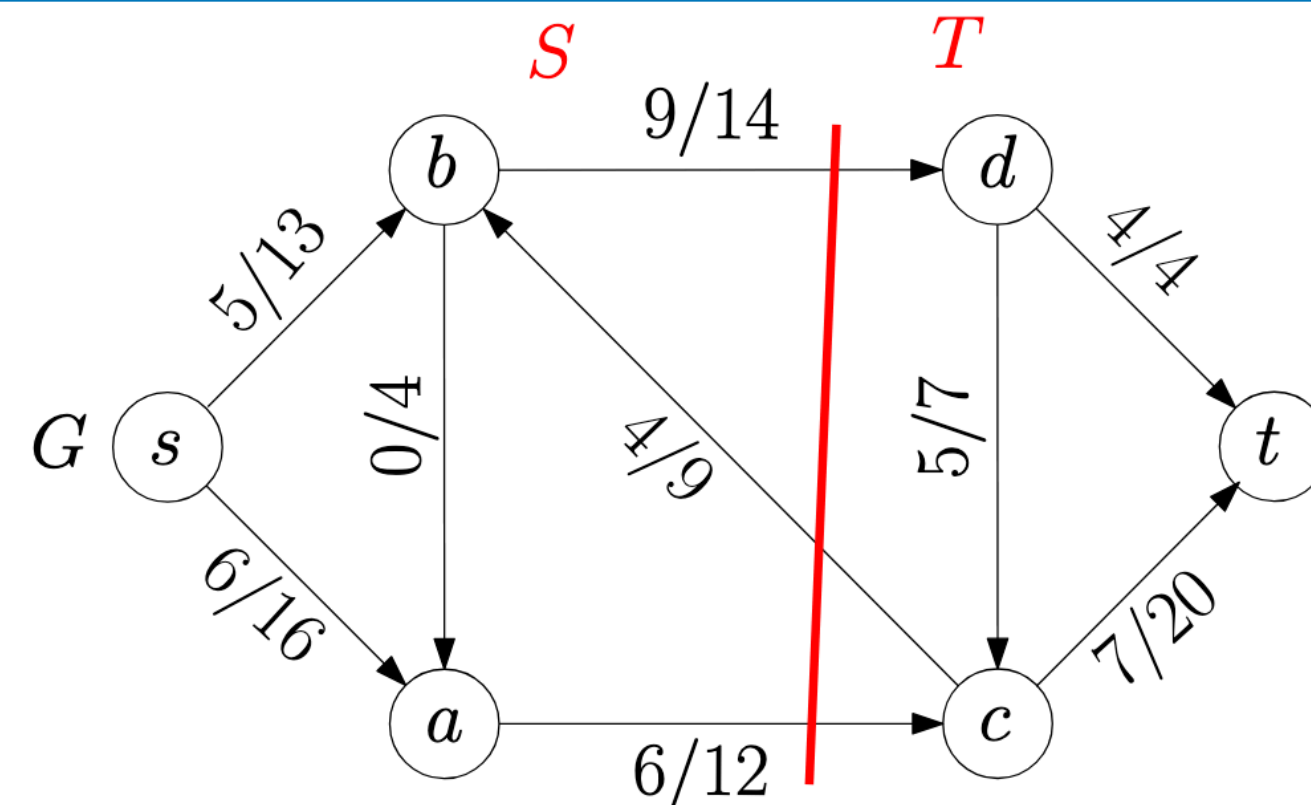
- (1) f is a maximum flow
- (2) There is no s - t path P in G^f with $\delta(P) > 0$
- (3) There is $S, \neg S \subseteq V$ such that $\text{cut}(S) = |f|$

Proof strategy:

(1) $\implies$ (2): proven by $\neg(2)\implies\neg(1)$

(2) $\implies$ (3): every edge is stuck

(3) $\implies$ (1): every flow is at most any cut



The following statements are equivalent:

- (1)** f is a maximum flow
- (2)** There is no s - t path P in G^f with $\delta(P) > 0$
- (3)** There is $S, \neg S \subseteq V$ such that $\text{Cut}(S) = |f|$

(2) $\Rightarrow$ (3): Let S be set of vertices reachable in G^f from s

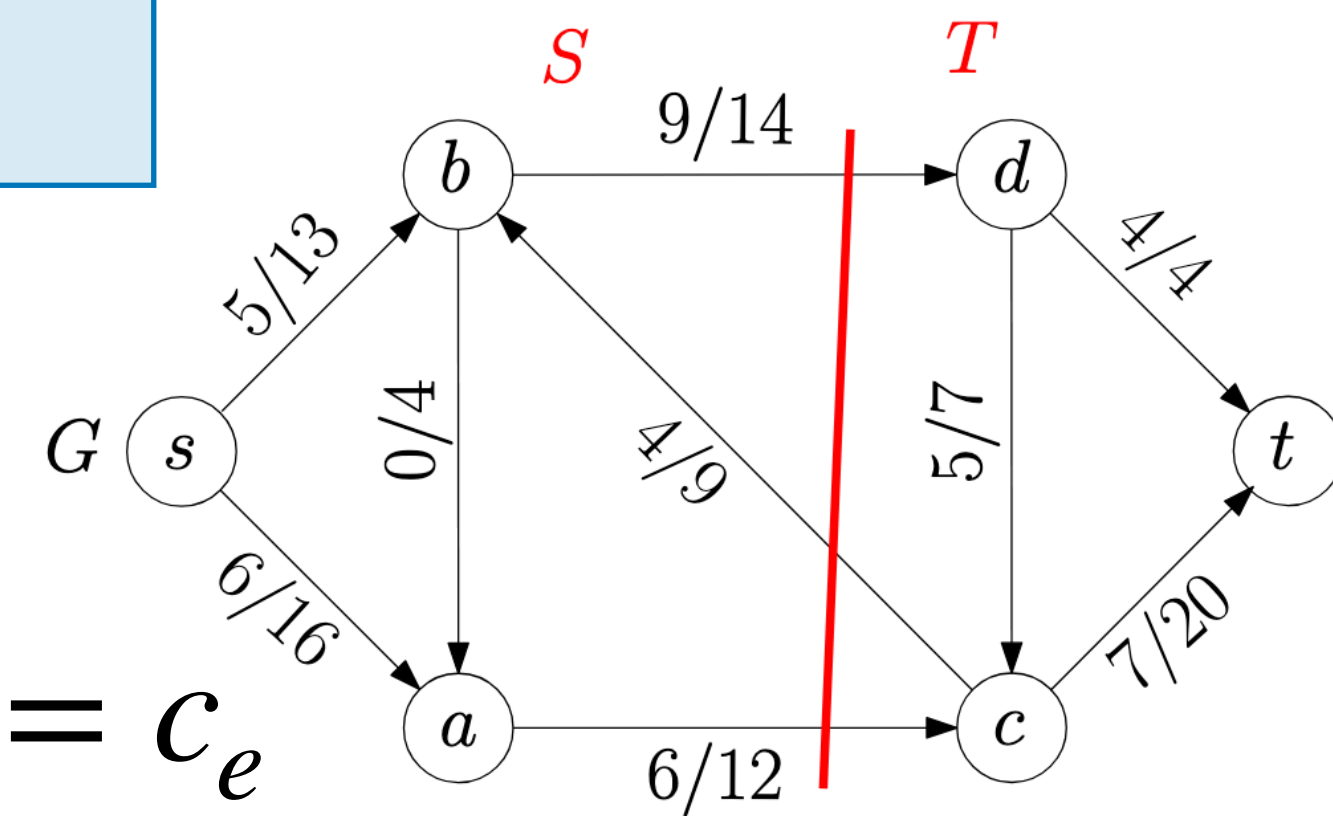
- Observation:

- Every edge $e = (a, b) \in E$ with $a \in S, b \in \neg S$, then $f_e = c_e$
- Every edge $e = (b, a) \in E$ with $a \in S, b \in \neg S$, then $f_e = 0$

- Otherwise b is reachable in G^f as well.

- By flow conservation:

$$|f| = \sum_{a \in S, b \notin S} f_{a,b} - \sum_{a \in S, b \notin S} f_{b,a} = \sum_{\substack{(a,b) \in E \\ a \in S, b \notin S}} c_{a,b} = \text{Cut}(S)$$

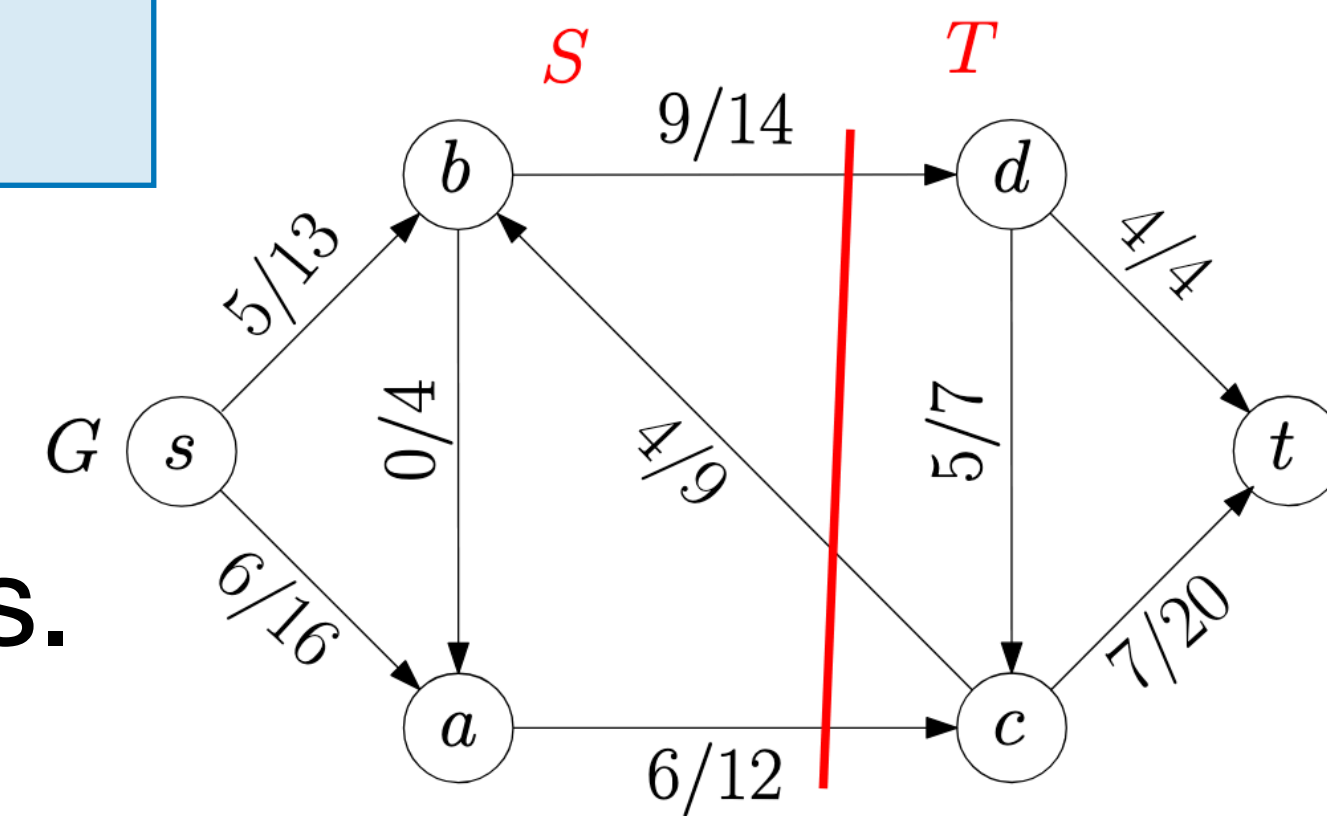


The following statements are equivalent:

- (1)** f is a maximum flow
- (2)** There is no s - t path P in G^f with $\delta(P) > 0$
- (3)** There is $S, \neg S \subseteq V$ such that $\text{Cut}(S) = |f|$

(3) $\Rightarrow$ (1): $|f| \leq \text{Cut}(S)$ for any f, S

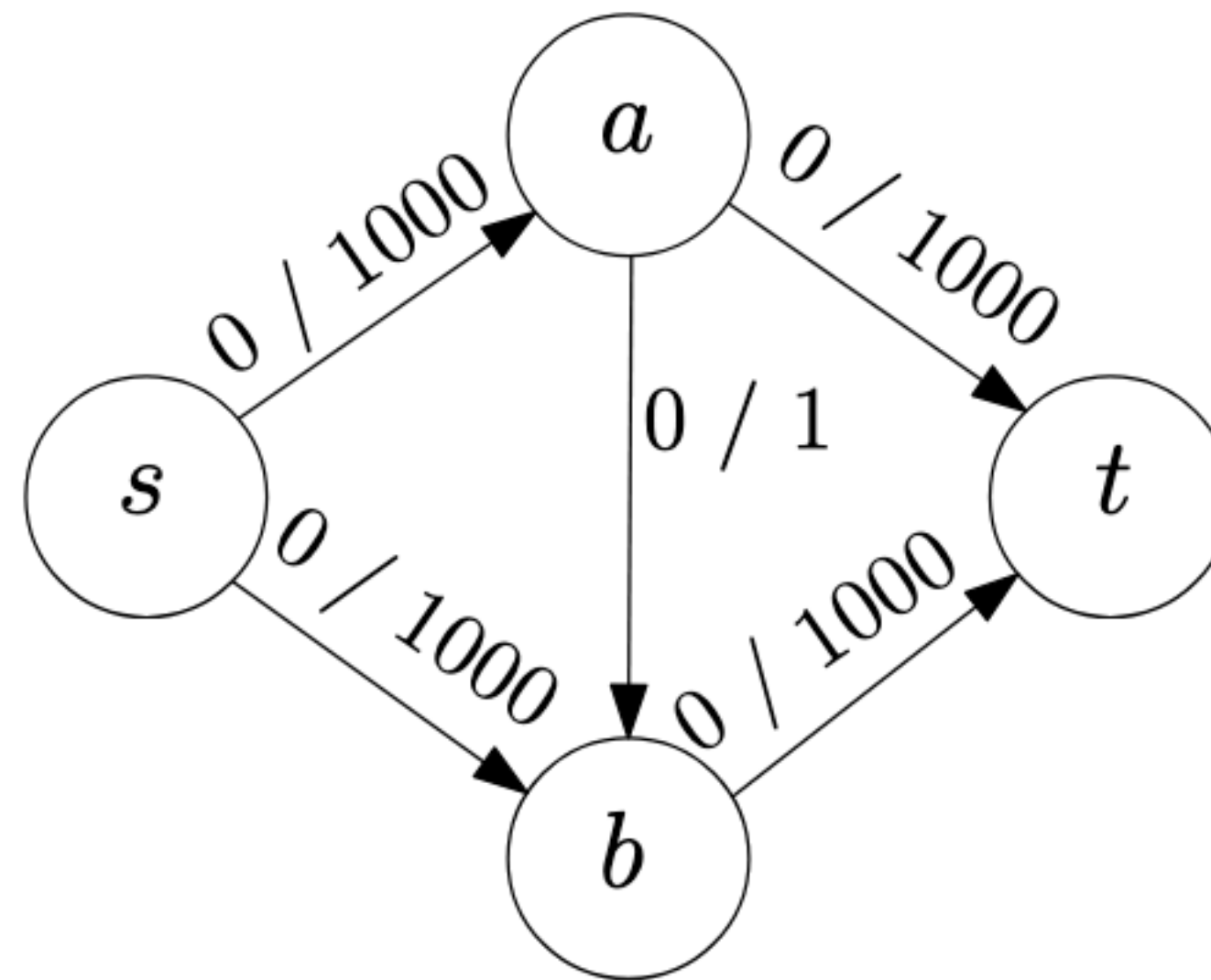
- Proof by decomposition: decompose f , keep only s - t paths.
- Consider a path P with p flow.
 - Pass from S to $\neg S$
 - Uses $\geq p$ capacity of $\text{Cut}(S)$ (backward is allowed)
- $|f| = \sum_P p \leq \text{Cut}(S)$



Augmenting path

Ford-Fulkerson: let $\delta(P)$ be residual capacity for path P
While exists s - t path $P \in G^f$ with $\delta(P) > 0$:
increase the flow f in P by $\delta(P)$

- Time cost? Worst case.

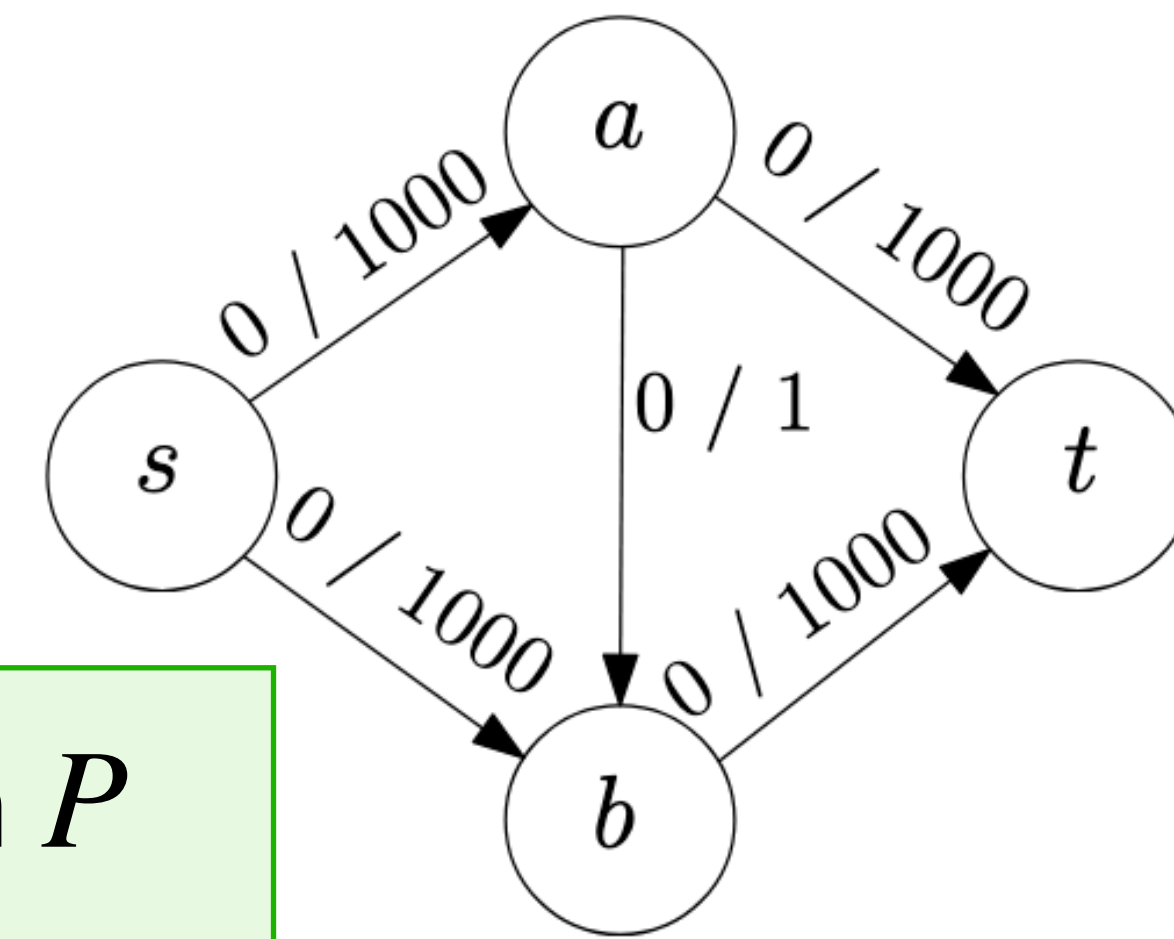


- Naive Ford-Fulkerson algorithms takes $O(|f^*| \cdot m)$ time

Augmenting path

Greedy

Ford-Fulkerson: let $\delta(P)$ be residual capacity for path P
While exists s - t path $P \in G^f$ with $\delta(P) > 0$:
increase the flow f in P by $\delta(P)$



- Theorem: Any s - t flow can be decomposed into cycles and s - t paths.
- Greedy idea: adopt the “fattest” augmenting path
- Max flow $|f^*| = |f| + |f'|$, where f' is the max flow in G^f
- By decomposition, exists a path P with flow $\geq |f'|/m$, since $\leq m$ paths
- Convergence: $|f'| \leq |f^*|(1 - 1/m)^t$ after t iterations.
 - #iterations = $O(m \log |f^*|)$. Might not converge for real capacity.

Augmenting path

Greed

Observation:
augmenting fattest path
results in **thinner** path

Ford-Fulkerson: let $\delta(P)$ be residual capacity for path P

While exists s - t path $P \in G^f$ with $\delta(P) > 0$:

increase the flow f in P by $\delta(P)$

- Greedy idea: adopt the “fattest” augmenting path
- Max flow $|f^*| = |f| + |f'|$, where f' is the max flow in G^f
- By decomposition, exists a path P with flow $\geq |f'|/m$, since $\leq m$ paths
- Convergence: $|f'| \leq |f^*|(1 - 1/m)^t$ after t iterations.
 - #iterations = $O(m \log |f^*|)$. Might not converge for real capacity.
- Finding “fat” paths: binary search + BFS in $O(m \log |f^*|)$ time

Augmenting path

Greedy + Scaling

Observation:
augmenting shortest path
results in **longer** path

Ford-Fulkerson: let $\delta(P)$ be residual capacity for path P
While exists s - t path $P \in G^f$ with $\delta(P) > 0$:
increase the flow f in P by $\delta(P)$

- Greedy idea: for $i = \lceil \log_2 C \rceil, \dots, 0$: while exists flow $|f| \geq 2^i$: augment
- Imagine residual graph $G_i^f = (V, E'', f)$ with $E'' \triangleq \{e \in E' : c_e \geq 2^i\}$
 - Any edge e is of capacity $c_e \in [2^i, 2^{i-1})$
 - All augmenting paths contribute $\geq 2^i$
 - #flows in G_i^f is $\leq m$ by decomposition
- Inner loop repeats $O(m)$ times. Time cost $O(\log_2 C \cdot m \cdot m)$

Augmenting path

Shortest augmenting [Dinitz, Edmonds-Karp]

Observation:
augmenting shortest path
results in **longer** path

Ford-Fulkerson: let $\delta(P)$ be residual capacity for path P

While exists s - t path $P \in G^f$ with $\delta(P) > 0$:

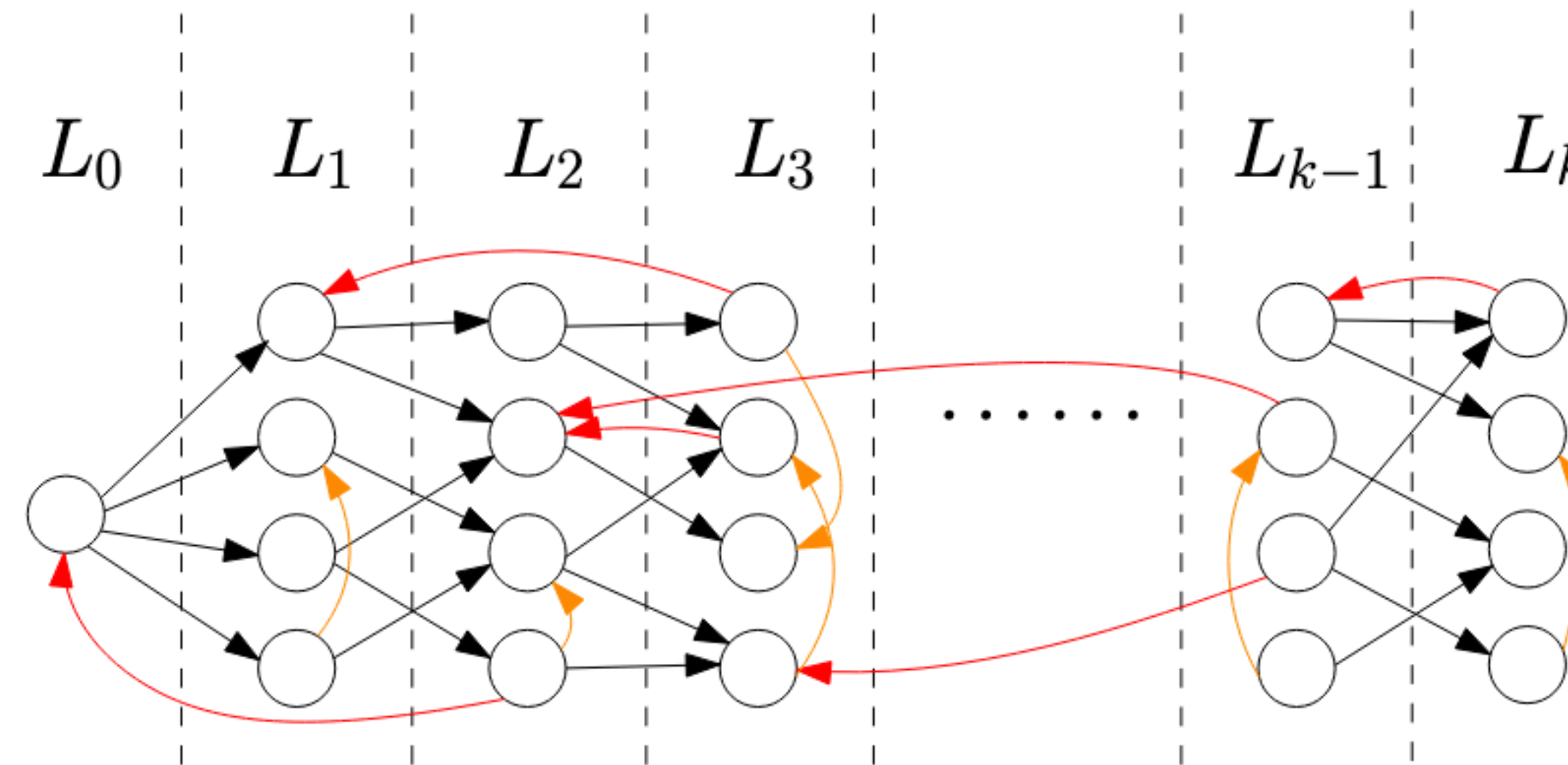
increase the flow f in P by $\delta(P)$

- Algorithm: keep finding shortest path, then augment
- Time cost?
 - Path finding: $O(m)$ time.
 - Path lengths: $1, \dots, n$.
 - #paths for each length: $\leq m$
- $O(m^2n)$ time in total

Augmenting path

Shortest augmenting [Dinitz, Edmonds-Karp]

Observation:
augmenting shortest path
results in **longer** path

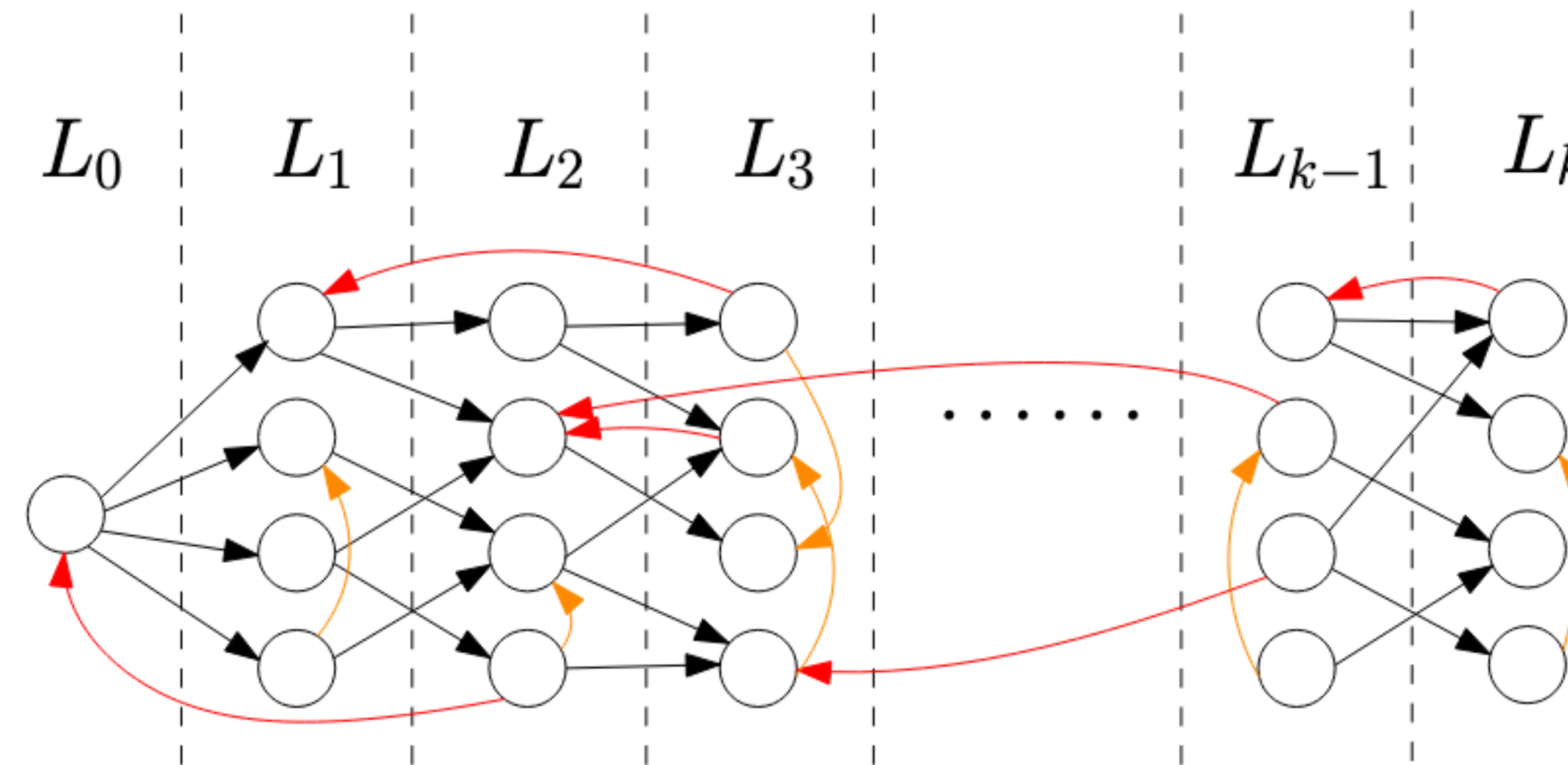


- Imagine the shortest $s-t$ paths in G^f by layers
 - Forth edges: edges between adjacent layers
 - Side edges: edges between the same layer
 - Back edges: edges from further layer to nearer layer
 - Jumping edge: edge from nearer layer to non-adjacent further layer

Augmenting path

Shortest augmenting [Dinitz, Edmonds-Karp]

Observation:
augmenting shortest path
results in **longer** path

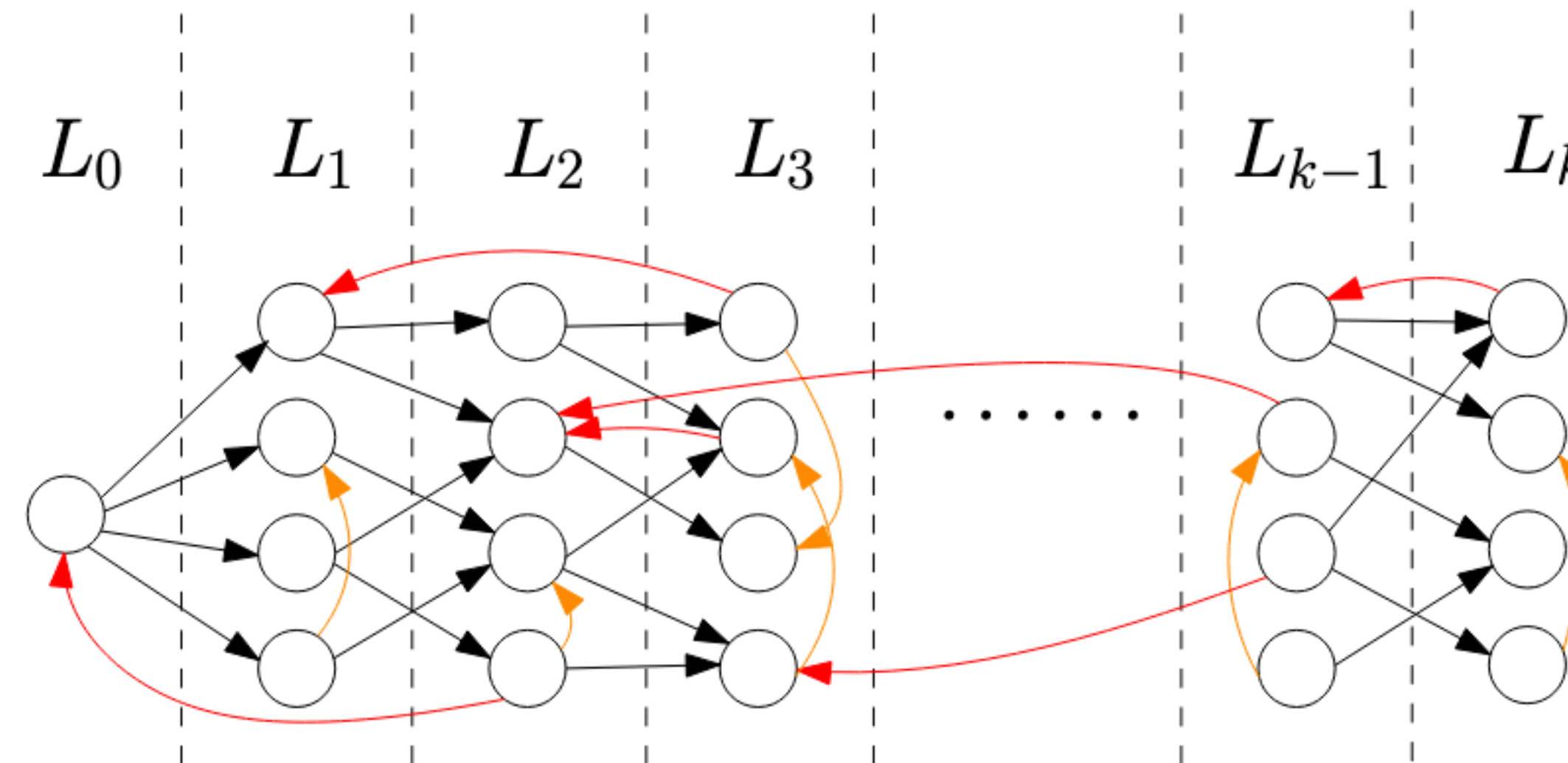


- Imagine the shortest $s-t$ paths in G^f by layers
- Augmenting removes ≥ 1 forth edges, and adds ≥ 0 back edges
- No jumping edge is added by augmenting

Augmenting path

Improving [Dinitz]

Observation:
augmenting shortest path
results in **longer** path

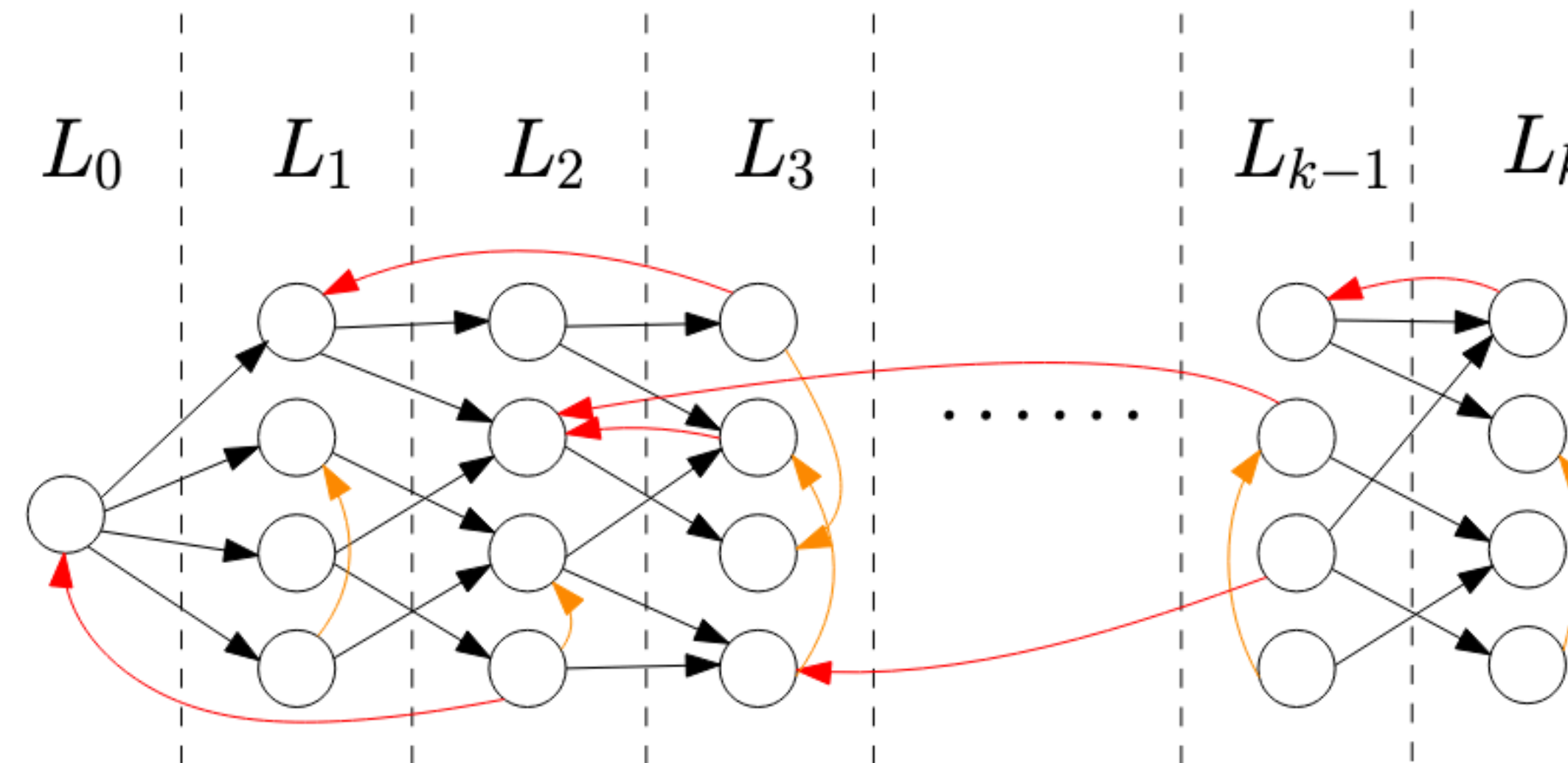


- Imagine the shortest $s-t$ paths in G^f by layers
- Augmenting removes ≥ 1 forth edges, and adds ≥ 0 back edges
- No jumping edge is added by augmenting
- Augmenting all shortest paths at one round: DFS only on forth edges.

Augmenting path

Improving [Dinitz]

Observation:
augmenting shortest path
results in **longer** path



- Augmenting all shortest paths at one round: DFS only on forth edges.
 - DFS: $O(nm)$ time.
 - Path lengths: $1, \dots, n$.
- $O(n^2m)$ time in total

Augmenting path

Algorithms	time	type
Ford-Fulkerson	$O(m^2 f)$	pseudo-polynomial
Scaling	$O(m^2 \ln f)$	wealy-polynomial
Edmonds-Karp Dinitz	$O(m^2 n)$ $O(mn^2)$	strongly-polynomial

Polynomial time: in proportion to polynomial of n , where n is the **size of input**

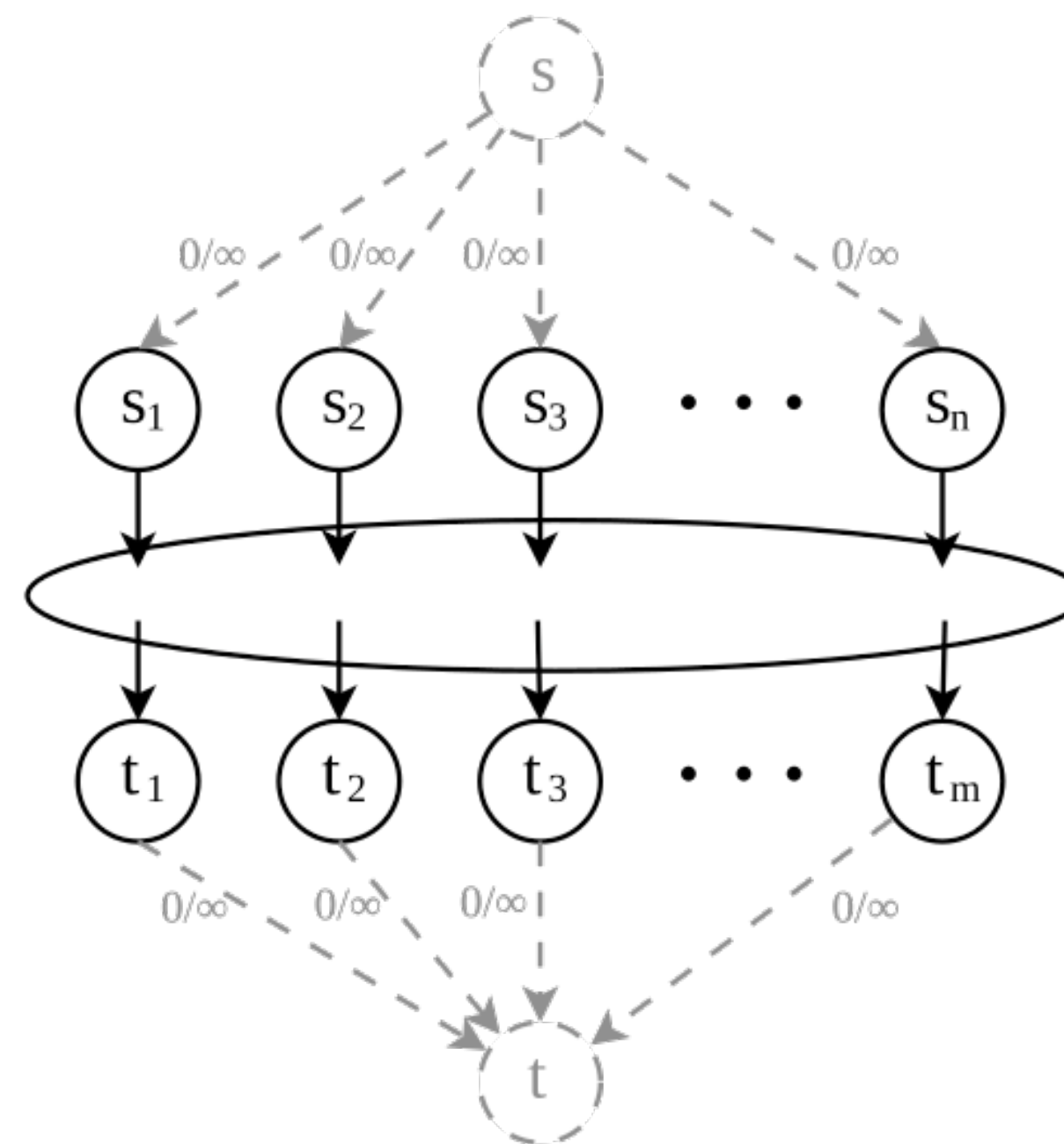
Maximum flow problem

Method	Year	Complexity
Linear programming		
Ford–Fulkerson algorithm	1955	$O(EU)$
Edmonds–Karp algorithm	1970	$O(VE^2)$
Dinic's algorithm	1970	$O(V^2E)$
MKM (Malhotra, Kumar, Maheshwari) algorithm ^[1]	1978	$O(V^3)$
Dinic's algorithm with dynamic trees	1983	$O(VE \log V)$
General push–relabel algorithm ^[2]	1986	$O(V^2E)$
Push–relabel algorithm with <i>FIFO</i> vertex selection rule ^[2]	1988	$O(V^3)$
Push–relabel algorithm with <i>maximum distance</i> vertex selection rule ^[3]	1988	$O(V^2\sqrt{E})$
Push-relabel algorithm with dynamic trees ^[2]	1988	$O\left(VE \log \frac{V^2}{E}\right)$
KRT (King, Rao, Tarjan)'s algorithm ^[4]	1994	$O\left(VE \log \frac{E}{V \log V} V\right)$
Binary blocking flow algorithm ^[5]	1998	$O\left(E \cdot \min\{V^{2/3}, E^{1/2}\} \cdot \log \frac{V^2}{E} \cdot \log U\right)$
James B Orlin's + KRT (King, Rao, Tarjan)'s algorithm ^[6]	2013	$O(VE)$
Kathuria-Liu-Sidford algorithm ^[7]	2020	$E^{4/3+o(1)}U^{1/3}$
BLNPSSSW / BLLSSSW algorithm ^{[8][9]}	2020	$\tilde{O}((E + V^{3/2}) \log U)$
Gao-Liu-Peng algorithm ^[10]	2021	$\tilde{O}(E^{\frac{3}{2}-\frac{1}{328}} \log U)$
Chen, Kyng, Liu, Peng, Gutenberg and Sachdeva's algorithm ^[11]	2022	$O(E^{1+o(1)} \log U)$
Bernstein, Blikstad, Saranurak, Tu ^[12]	2024	$O(n^{2+o(1)} \log U)$

Multi-Source Multi-Sink

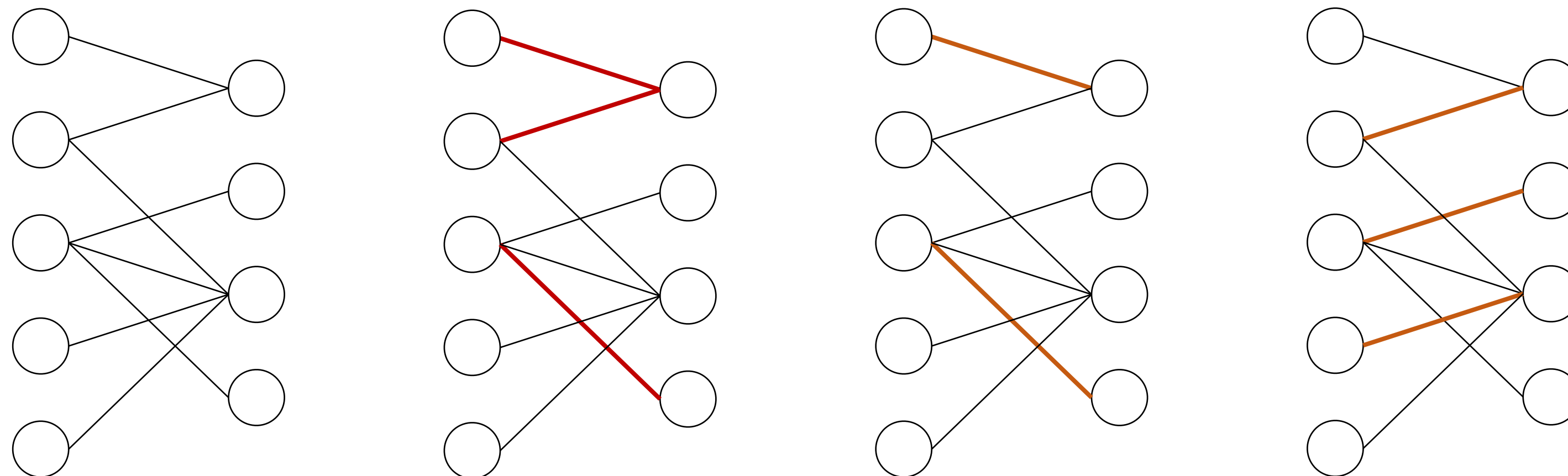
Input: A network $G = (V, E)$ and capacity $c : E \rightarrow \mathbb{R}^+$, with sources $S = \{s_1, \dots, s_n\}$ and sinks $T = \{t_1, \dots, t_m\}$.

Output: A maximum flow from S to T across G .



Maximum Bipartite Matching

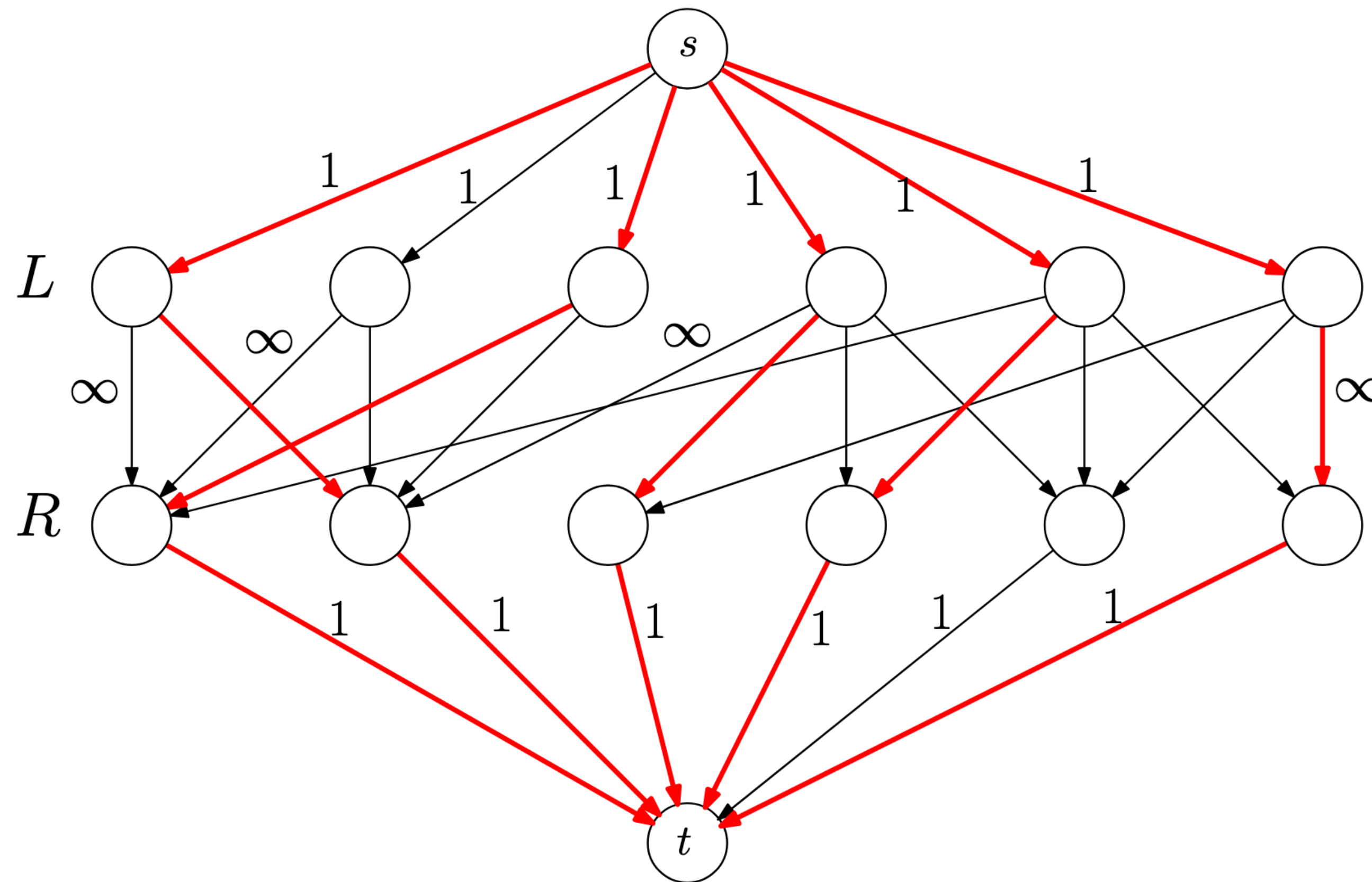
- A graph $G = (V, E)$ is **bipartite** if V can be partitioned into L and R , such that each edge connects a vertex in L and a vertex in R .
- A **matching** of a graph $G = (V, E)$ is a subset $M \subseteq E$, such that for each $v \in V$, at most one edge in M is incident on v .
- A **maximum matching** is a matching of maximum cardinality.
- **Problem:** Find a maximum matching of a bipartite graph.



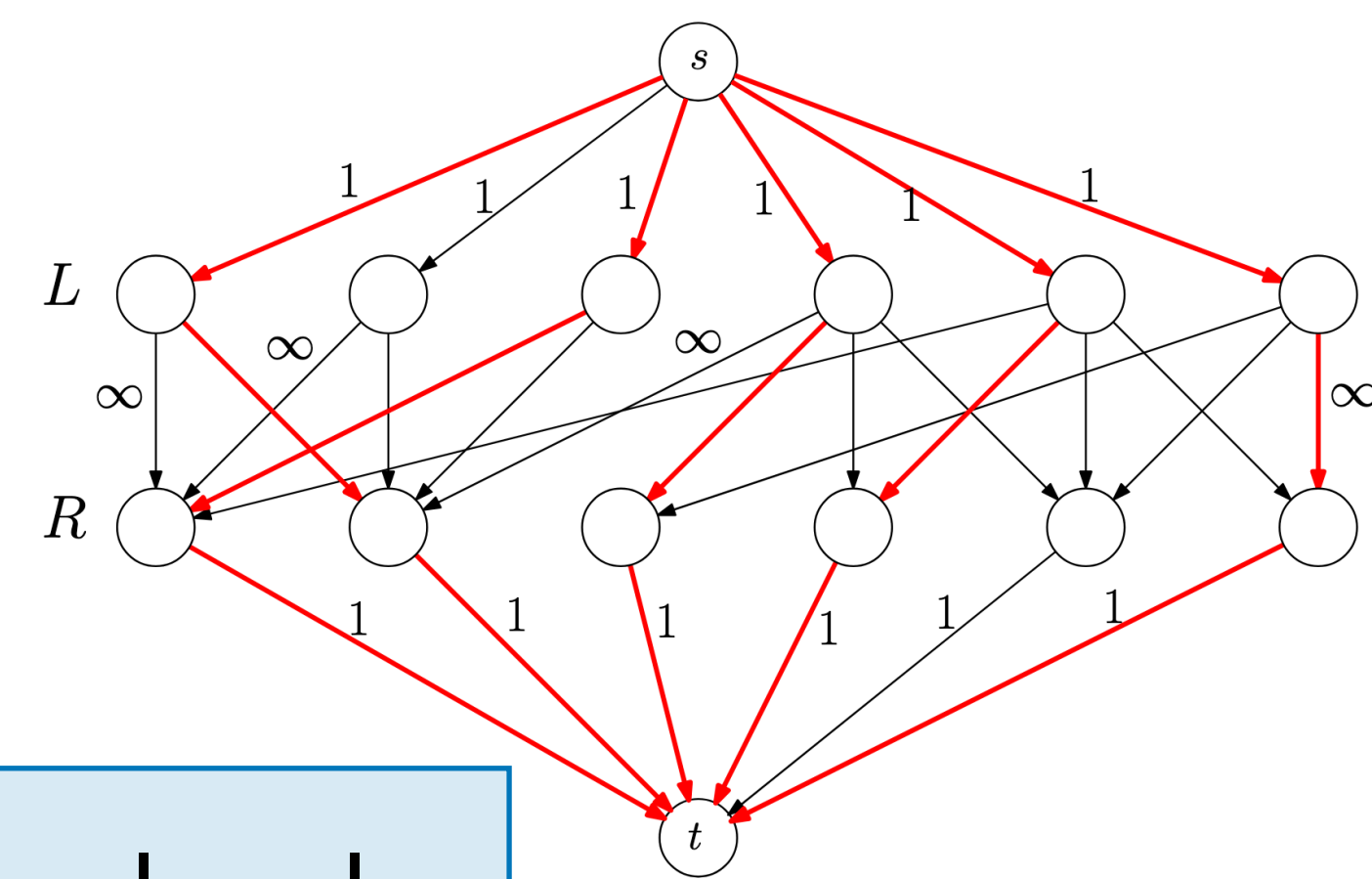
Maximum Bipartite Matching

Input: A bipartite graph $G = (L \cup R, E)$.

Output: A maximum matching of a bipartite graph.



Maximum Bipartite Matching

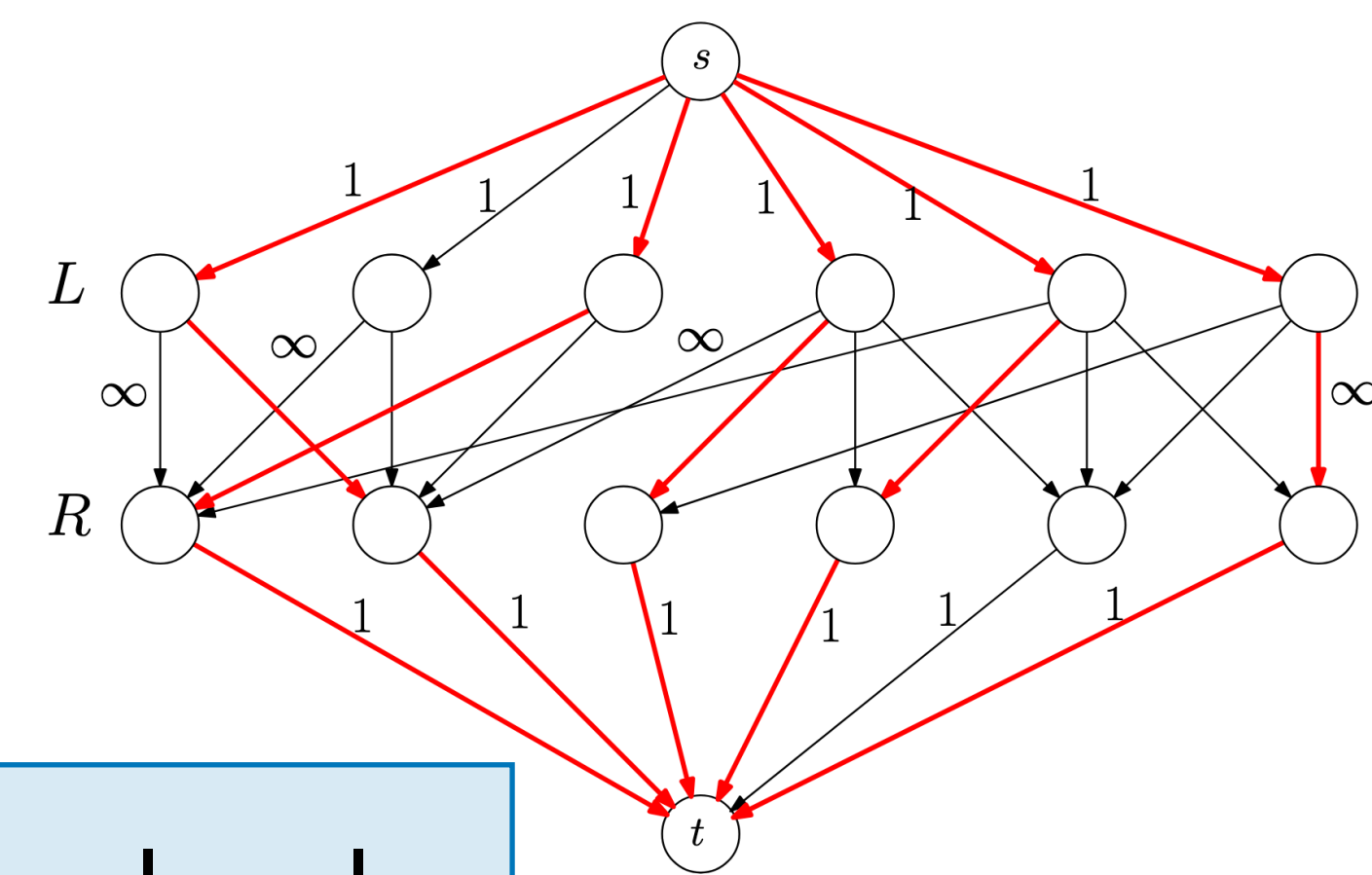


- (1) $\exists$ matching M of $G \Rightarrow \exists$ flow f of G' with $|f| = |M|$
- (2) $\exists$ flow f of $G' \Rightarrow \exists$ matching M of G with $|M| = |f|$.

(1):

- For each $e = (u, v) \in M$, let $f_{su} = f_e = f_{vt} = 1$
- For each $e = (u, v) \notin M$, let $f_e = 0$
- f is valid, and $|f| = |M|$

Maximum Bipartite Matching



- (1) $\exists$ matching M of $G \Rightarrow \exists$ flow f of G' with $|f| = |M|$
- (2) $\exists$ flow f of $G' \Rightarrow \exists$ matching M of G with $|M| = |f|$.

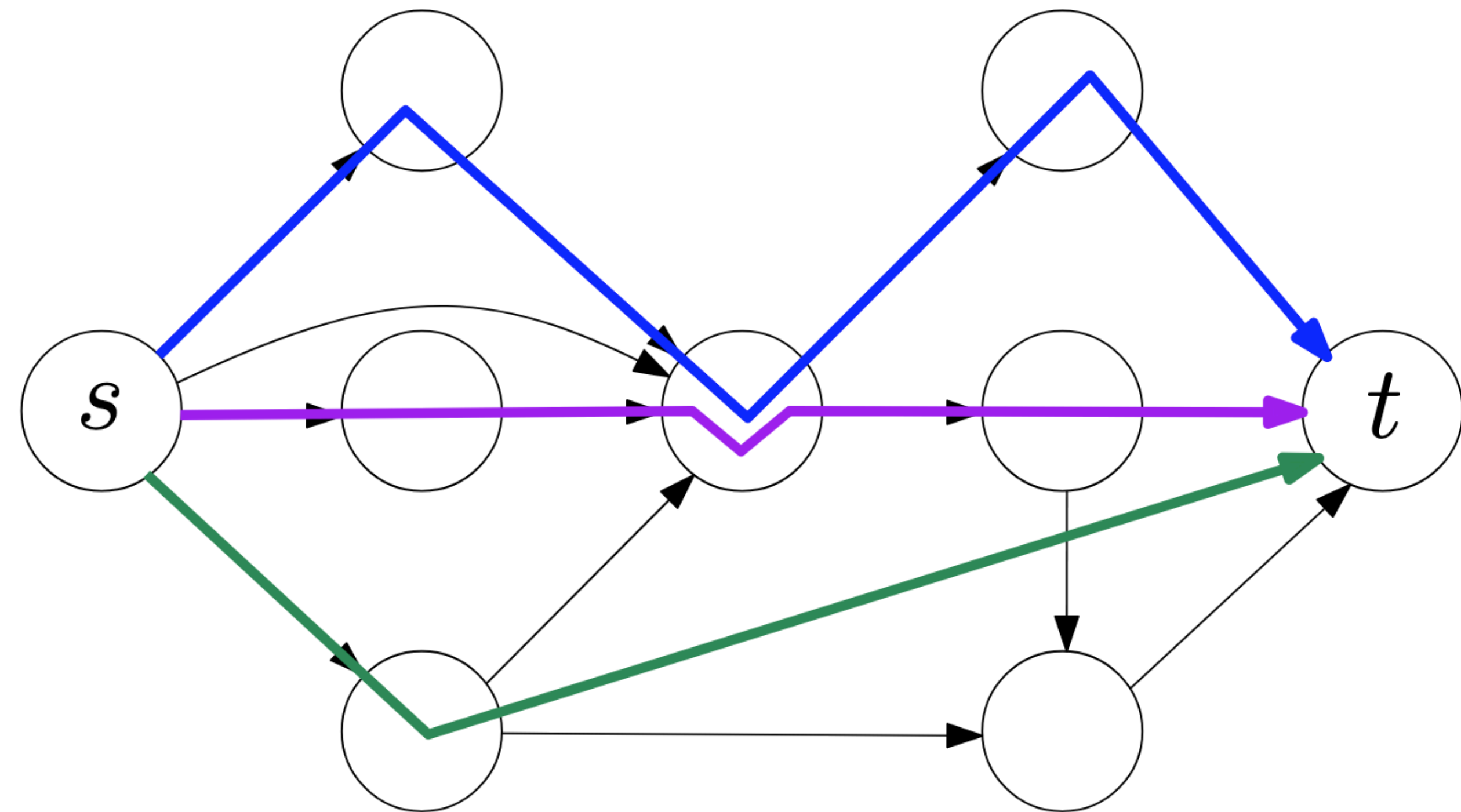
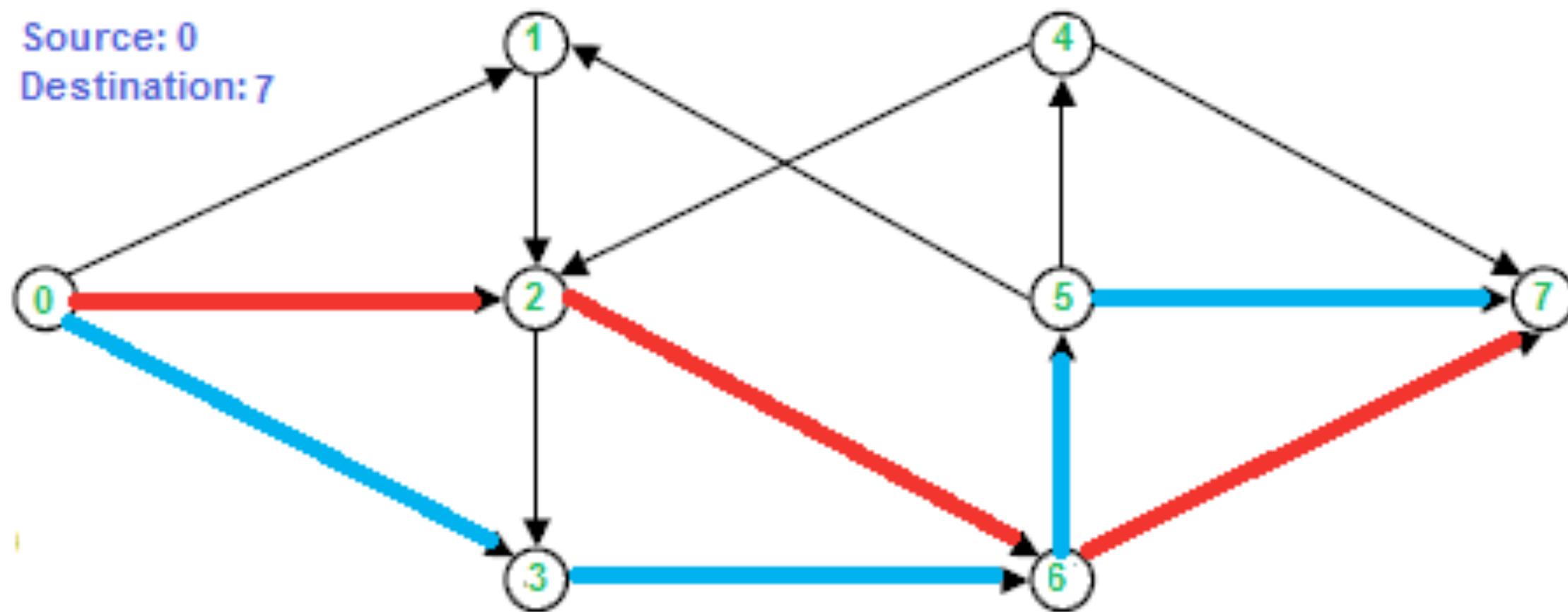
(2): Let $M \triangleq \{(u, v) \in E : u \in L, v \in R, f_e > 0\}$

- M is a matching:
 - for any $u \in L$, $\sum_{v \in \delta(u) \setminus s} f_{uv} \leq 1$; for any $v \in R$, $\sum_{u \in \delta(v) \setminus t} f_{uv} \leq 1$.
 - Any $u \in L$ matches with at most one $v \in R$, so does any $v \in R$.
- $|M| = |f|$

#disjoint Paths

Input: A digraph $G = (V, E)$ and source & sink $s, t \in V$.

Output: The maximum **#edge-disjoint** paths from s to t



s - t Min-Cut

Input: A digraph $G = (V, E)$ and weights $c : E \rightarrow \mathbb{R}^+$, and source & sink $s, t \in V$.

Output: The minimum cut whose removal disconnects s, t .

The following statements are equivalent:

- (1) f is a maximum flow
- (2) There is no s - t path P in G^f with $\delta(P) > 0$
- (3) There is $S, \neg S \subseteq V$ such that $\text{cut}(S) = |f|$

Global Min-Cut

Input: A digraph $G = (V, E)$ and weights $c : E \rightarrow \mathbb{R}^+$.

Output: The minimum cut whose removal disconnects G

The following statements are equivalent:

- (1) f is a maximum flow
- (2) There is no s - t path P in G^f with $\delta(P) > 0$
- (3) There is $S, \neg S \subseteq V$ such that $\text{cut}(S) = |f|$

Maximum Flow is P -complete

Under log space reduction

